



REACT

VIBE CODING

COURSE SLIDES · NON-WSQ

AI Vibe Coding for React Development

Course Code: C1143

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Trainer: Dr. Alfred Ang

Version v4.0 · 5 October 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE

Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

CERTIFICATION

Full-stack web development, AI-assisted engineering and cloud.

DELIVERS

Courses on React, vibe coding, AI agents and software engineering.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

- Your name and organisation / role.
- Your experience with JavaScript, React or any AI coding assistant.
- What you want to build after this course.

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

One conversation at a time.

5

Be punctual; return from breaks on time.

6

Attend both days to get the most from the course.

SCHEDULE

Lesson Plan — 2 Days, 7.5 hours/day

Day 1

- Day 1 — Vibe Coding, Cloud Deployment & Core React
 - Topic 1: Build a React App with Vibe Coding (Labs 1.1–1.3)
 - Topic 2: Deploy to the Cloud (Labs 2.1–2.3)
 - Topic 3: Core React Concepts (Labs 3.1–3.4)

Day 2 & timing

- Day 2 — Hooks, Data, Routing & Capstone
 - Topic 4: React Hooks (Labs 4.1–4.5)
 - Topic 5: APIs & Neon Database (Labs 5.1–5.4)
 - Topic 6: React Router & Deployment (Labs 6.1–6.5)
 - Daily timing: 9:30am–5:30pm · 30-min lunch

Access the Hands-On Labs

Option A — Download the courseware

- Sign in to the LMS and download the course pack.
- Unzip it — every lab lives in labs/topic-N/.

Option B — Build it yourself

- `npm create vite@latest cookbake -- --template react`
- Each lab folder holds a drop-in src/ snapshot.

Courseware on the LMS:

<https://lms-tms.tertiaryinfotech.com/>

Every lab folder is a checkpoint: if an AI edit breaks your app, copy that lab's src/ over your own and carry on.



CORE CONCEPTS

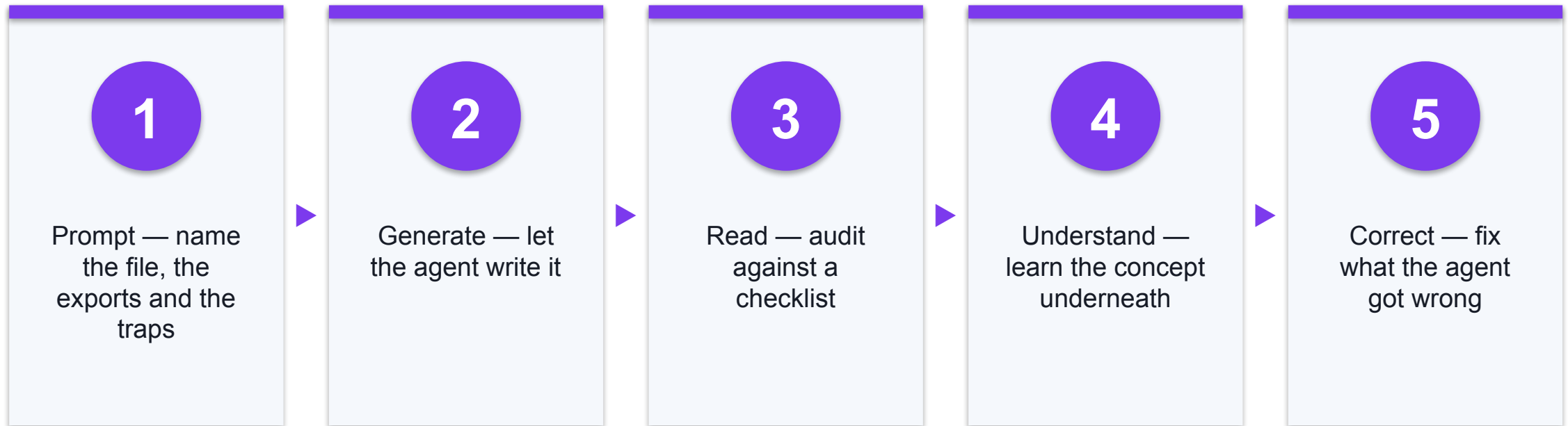
Vibe Coding & the React Model

WHY THIS COURSE EXISTS

The AI writes the code. You supply the judgement.

An agent will hand you working React in seconds — and a `key={index}` that corrupts your list on delete. If you cannot read it, you cannot ship it.

The Vibe Coding Loop



The React Mental Model

1

A component is a function

It takes props and returns JSX describing the UI for the current state.

2

JSX is not HTML

It compiles to function calls returning plain JavaScript objects.

3

State drives the UI

You change state; React works out the minimum DOM edit to match.

4

The Virtual DOM

An in-memory object tree. Cheap to build, cheap to diff — the real DOM is what's slow.

5

Props flow down

Parent to child, read-only. A child calls a callback instead of writing to a prop.

6

Hooks add memory

useState remembers, useEffect synchronises with the outside world.

The Stack You Will Build On

Front end

- Vite — dev server & build
- React 19 — function components
- React Router 7 — navigation

Back end

- Vercel Functions — the /api tier
- Neon — serverless Postgres
- bcrypt + JWT — accounts & auth

Ship it

- Git & GitHub — your safety net
- Vercel — deploy on every push
- An AI agent — Claude Code / Cursor

The Project: Cook & Bake Academy

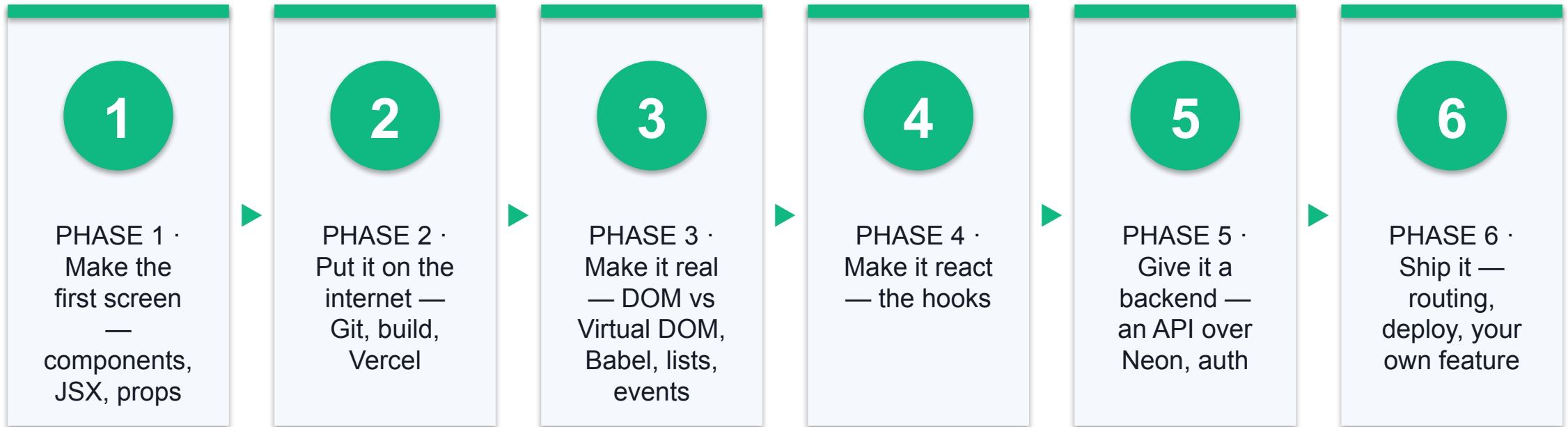
The app

- What it is
 - A cooking & bakery school website
 - 20 courses, enrolments, reviews
- Day 1
 - Vibe-coded landing page, deployed
 - Refactored into real components

How it grows

- Day 2
 - Made interactive with hooks
 - A serverless API over Neon Postgres
 - Accounts, a private dashboard, routing
 - Deployed live to Vercel + Neon

The Build — One App, Six Phases



THE ONE SECURITY RULE

Security is not hiding a key. It is trusting the token, not the request.

A VITE_ variable is compiled into the bundle every visitor downloads, so DATABASE_URL lives only on the server. The user id comes from the verified JWT — never from the request body.

FOUNDATIONS

How React Really Renders

The real DOM · Updating a page without React · The Virtual DOM · Babel & JSX

BEFORE WE START

The page you see IS the DOM. React just decides how to change it.

You already know JavaScript. What makes React click is one layer down: what the DOM is, why touching it by hand hurts, and how Babel turns your JSX into plain objects React can diff.

The DOM is the browser's object tree

- When the browser parses HTML it builds the DOM: a live tree of objects, one per element.
- The page you see on screen is a picture of that tree. Change the tree and the screen changes.
- JavaScript can reach into it: `document.getElementById` returns the real object for one node.
- Every element node has properties you can write — `textContent`, `className`, `style`, `value`.
- React does not replace the DOM. It manages it for you, so you stop writing the code below.

REMEMBER The DOM is the browser's live object tree — the page itself, not a copy.

```
// The DOM is a real, live object you can hold:  
const el = document.getElementById('count')
```

```
el.textContent = '19'      // the screen updates  
el.className = 'badge is-on' // so does this
```

THINK OF IT LIKE...

The DOM is not a picture of the page — it IS the page, a live model you can reach in and edit.

Updating the page without React

- Without React you must say HOW to update: find the node, then overwrite each property by hand.
- You write the change twice — once in the markup, once in the patch code — and they drift apart.
- The data now lives in a DOM string. Reading '19' back out and adding to it is your problem.
- Add a filter, a search box and a shortlist and every one of them must patch every affected node.
- Twenty course cards, four fields each: eighty little patches you have to remember to keep in step.

REMEMBER Hand-patching the DOM means saying HOW, everywhere, forever — and it does not scale.

```
// Vanilla JS: filter the catalogue by hand.  
function showBakery() {  
  const cards = document.querySelectorAll('.card')  
  cards.forEach((card) => {  
    const isBakery = card.dataset.category === 'Bakery'  
    card.style.display = isBakery ? 'block' : 'none'  
  })  
  document.getElementById('count').textContent = '10 courses'  
  // ...and now update the chip, the heading, the empty  
  // state, the URL... every one, every time, by hand.  
}
```

THINK OF IT LIKE...

Manual DOM work is giving turn-by-turn directions from memory — miss one turn and nobody notices until you are lost.

The Virtual DOM is React's plan

- The Virtual DOM is a lightweight tree of plain JavaScript objects describing what the UI should be.
- It is cheap: making one is just allocating objects. Touching the REAL DOM is what costs.
- On every render React builds a fresh virtual tree for the current state.
- Reconciliation is the diff: it compares the new tree with the previous one, node by node.
- It then patches ONLY the real DOM nodes that actually changed, and leaves the rest alone.

REMEMBER React diffs a cheap JS tree, then patches only the real nodes that changed.

```
// State changes: 20 courses -> 10 (Bakery only).  
// React builds a NEW virtual tree, diffs it, and  
// concludes: remove 10 <article> nodes, and set one  
// text node from '20 courses' to '10 courses'.  
//  
// It does NOT rebuild the navbar, the hero, the  
// footer or the surviving cards. They did not change.
```

THINK OF IT LIKE...

The Virtual DOM is an architect's drawing — React compares the new plan to the old before knocking down a single real wall.

Real DOM vs Virtual DOM, side by side

- Imperative (no React): you locate nodes and mutate them. You own every step of HOW.
- Declarative (React): you describe WHAT the UI is for the current state. React works out the steps.
- The React version has no `getElementById`, no `textContent` and no `className` assignment anywhere.
- Because the UI is recomputed from state every time, it can never silently drift out of sync.
- Your job shrinks to one thing: keep the state correct, and describe the UI as a function of it.

REMEMBER Stop describing HOW to change the page; describe WHAT it should be.

```
// WITHOUT React — you patch the real DOM yourself:
const el = document.getElementById('seats')
el.textContent = String(seats)
el.className = seats > 0 ? 'badge' : 'badge is-full'

// WITH React — you describe the result, once:
<span className={seats > 0 ? 'badge' : 'badge is-full'}>
  {seats}
</span>
```

THINK OF IT LIKE...

Imperative is reciting the recipe step by step; declarative is ordering the dish and letting the kitchen work out the steps.

Babel: the browser never sees JSX

- JSX is not valid JavaScript. Paste it into a browser console and you get a syntax error.
- Babel is a compiler that rewrites your JSX into ordinary function calls before it ships.
- Vite runs Babel for you through `@vitejs/plugin-react` — it is why the plugin is in `vite.config.js`.
- `<CourseCard fee={680}/>` becomes `React.createElement(CourseCard, { fee: 680 })`.
- That call is not markup and it is not a DOM node. It returns a plain JavaScript OBJECT.

REMEMBER JSX is sugar. Babel turns every tag into a call that returns a plain object.

```
// YOU WRITE (JSX):  
<CourseCard course={course} fee={680} />  
  
// BABEL COMPILES IT TO (plain JavaScript):  
React.createElement(CourseCard, { course: course, fee: 680 })  
  
// WHICH RETURNS (a plain object — this is a React element):  
{ type: CourseCard, props: { course: {...}, fee: 680 } }
```

THINK OF IT LIKE...

Babel is the translator in the booth — you speak JSX, the browser only ever hears JavaScript.

Why that one fact explains everything

- You write `className`, not `class`, because `class` is a reserved word in JavaScript.
- Attributes are camelCase (`onClick`, `htmlFor`) because they are really keys in a props object.
- `{}` is a window for an `EXPRESSION` — a value. An `if` or a `for` produces no value, so it cannot fit.
- One root element per return, because a JavaScript function returns exactly one value.
- Capitalised names, because `createElement('div')` means a tag but `createElement(CourseCard)` means YOUR function.

REMEMBER Every JSX rule falls out of one fact: it compiles to `createElement` calls.

```
<h3 className="card__title">{course.title}</h3>  
// -> createElement('h3', {className:'card__title'}, ...)  
// lowercase string 'h3' = a real HTML tag
```

```
<CourseCard course={course} />  
// -> createElement(CourseCard, {course})  
// capitalised IDENTIFIER = your function
```

THINK OF IT LIKE...

Once you know JSX is JavaScript in a costume, its 'weird' rules stop being rules and become consequences.

PHASE 01

Make the First Screen

Building Cook & Bake Academy — we add a vibe-coded landing page — the Cook & Bake navbar, hero and a grid of course cards

Make the First Screen

THE STORY SO FAR

You have an empty folder and a client brief: a website for a cooking & bakery school.

WHAT WE BUILD NOW

We add a vibe-coded landing page — the Cook & Bake navbar, hero and a grid of course cards.

THE IDEAS THIS PHASE NEEDS

To build it, you will learn components, JSX and props.

Vibe coding is a loop, not magic

- The loop is Prompt, Generate, Read, Understand, Correct — the agent writes, you supply the judgement.
- An AI agent produces working React in seconds, but 'working' and 'correct' are not the same thing.
- The value you add is reading the output against a checklist and knowing the concept underneath it.
- Every lab in this course repeats the same five steps so the habit becomes automatic.
- By the end you should build fast and be able to say exactly why every line is there.

REMEMBER Speed comes from the agent; correctness comes from you reading it.

```
// The five-step loop, every lab:  
// 1 Prompt 2 Generate 3 Read  
// 4 Understand 5 Correct  
//  
// The agent writes. You supply the judgement.
```

THINK OF IT LIKE...

Vibe coding is flying with autopilot on — you still watch the instruments and take the controls the moment something drifts.

A component is just a function

- A React component is a JavaScript function that returns JSX describing a piece of the UI.
- Its name must start with a capital letter, or Babel compiles it to a literal HTML tag instead.
- You use it by writing it like a tag: `<CourseCard />` calls the function and renders what it returns.
- Because it is a function, you reuse it by calling it many times, each with different props.
- This course uses function components and hooks only — never class components.

REMEMBER Capitalised function in, JSX out — that is the whole idea.

```
function CourseCard() {  
  return <h3>Artisan Sourdough Bread Baking</h3>  
}  
  
<CourseCard />
```

THINK OF IT LIKE...

A component is a cookie cutter: define the shape once, stamp out as many cookies as you like.

What a component really is

- A component is a plain function: it takes props in and returns one JSX tree describing the UI.
- It must return a single root — one tree — because a JavaScript function returns exactly one value.
- A component should be pure: given the same props, it must always return the same output.
- Purity means no surprises in render — no fetching, timers or mutation; those belong in effects.
- Purity is what lets React render it as often as it likes, and trust the result every time.

REMEMBER Same props in, same tree out — a capitalised, pure function.

```
function CourseCard({ title }) {  
  return <h3>{title}</h3> // one tree, no side effects  
}  
  
// same title in -> same markup out, every time
```

THINK OF IT LIKE...

A component is a vending machine: press the same buttons (props) and the same snack (UI) always drops out.

JSX is a blueprint, not HTML

- JSX looks like HTML but is not — Babel compiles it to `createElement` calls that return objects.
- Those objects describe what the UI should look like; React turns them into real DOM nodes for you.
- Because it is really JavaScript, HTML's `class` becomes `className` and attributes become `camelCase`.
- Understanding that JSX is code, not markup, explains every rule that follows in this topic.
- You never call `createElement` by hand — Vite runs Babel — but knowing it is there demystifies JSX.

REMEMBER JSX compiles to `createElement` calls returning plain JS objects, not HTML.

```
const el = <h1 className="brand">Cook & Bake</h1>
```

// Babel compiles it to:

```
React.createElement('h1', { className: 'brand' },  
  'Cook & Bake')
```

THINK OF IT LIKE...

JSX is a blueprint, not the building — React reads it and constructs the real DOM from the instructions.

One root element and Fragments

- A component must return a single root element, because a function returns exactly one value.
- Wrapping siblings in an extra `<div>` works but litters the page with meaningless containers.
- A Fragment (`<>...</>`) groups siblings under one root while adding no node to the real DOM.
- Reach for a Fragment whenever a wrapper div would only exist to satisfy this one-root rule.
- Returning two adjacent tags with no wrapper is a syntax error the agent will sometimes produce.

REMEMBER Return one root; use a Fragment to group without an extra div.

```
// RootLayout.jsx — the real app shell
return (
  <>
    <Navbar />
    <main><Outlet /></main>
    <Footer />
  </>
) // one root; <> adds no extra div
```

THINK OF IT LIKE...

One root is a single moving box React hands back — not an armful of loose items; a Fragment is a box with no cardboard.

Curly braces embed expressions

- Inside JSX, `{}` drops the result of a JavaScript expression into the output.
- An expression produces a value: a variable, a call, a ternary, a template literal all qualify.
- A statement such as `if`, `for` or a variable declaration does not — it produces no value and errors.
- This is why you use a ternary or `&&` inside JSX rather than an `if` statement.
- The braces run the code every render, so what you see always reflects the current data.

REMEMBER `{}` takes expressions (values), never statements like `if` or `for`.

```
<h1>Welcome to {academy}</h1>           // expression: ok
<span>S${course.fee}</span>               // expression: ok
<span>{isBakery ? '🍰 Bakery' : '👨🍳 Cooking'}</span> // ok

<h1>{if (x) {}}</h1>                     // statement: error
```

THINK OF IT LIKE...

Curly braces are a window cut into your markup — JavaScript shows through, but only values, never statements.

Props are a component's arguments

- A prop is a value passed into a component, exactly as you pass an argument to a function.
- You read props by destructuring them in the parameter list: `function CourseCard({ course })`.
- Give a default with `=` so a missing prop has a sensible value: `({ fee = 0 })`.
- Props flow one way, parent to child, and they are read-only — a child must never reassign a prop.
- String props use quotes; any other type uses braces: `title="Macaron Masterclass" fee={420}`.

REMEMBER Props are read-only arguments that flow down from parent to child.

```
function CourseCard({ title, fee = 0 }) {  
  return <h3>{title} — S${fee}</h3>  
}  
  
<CourseCard title="Macaron Masterclass" fee={420} />
```

THINK OF IT LIKE...

Props are a recipe card handed to a component: it reads the ingredients and cooks, but never edits the card.

One object prop beats nine loose ones

- The real CourseCard takes ONE prop — course — and destructures the fields it needs inside.
- That object's shape is the database row you will fetch in Topic 5, so nothing downstream changes.
- Nine separate props means nine things to keep in step at every call site. One object means one.
- Destructure inside the body when the object is the prop: `const { slug, title, fee } = course`.
- Collect leftover props with `...rest` and spread them onward: `<article {...rest}>`.

REMEMBER Pass the whole object; its shape is the DB row, so the card never changes.

```
// src/components/CourseCard.jsx (real code)
export default function CourseCard({ course }) {
  const { slug, title, category, level, fee } = course
  return <Link to={` /courses/${slug}`}>{title}</Link>
}
```

```
<CourseCard course={course} />
```

THINK OF IT LIKE...

Passing one course object is handing over the whole recipe card, not reading out nine ingredients down the phone.

The entry point: #root and main.jsx

- index.html is almost empty — its job is to provide one `<div id="root"></div>`.
- main.jsx is the entry file: it imports App and mounts it into that root div.
- `createRoot(...).render(<App/>)` is the single call that hands the whole app over to React.
- This is the ONE place React touches the real DOM by hand. Everything below it is declarative.
- Knowing this chain explains where the app 'starts' when you read AI-scaffolded code.

REMEMBER main.jsx mounts `<App/>` into the single `#root` div in index.html.

```
// index.html
<div id="root"></div>

// src/main.jsx (real code)
createRoot(document.getElementById('root')).render(
  <StrictMode><App /></StrictMode>,
)
```

THINK OF IT LIKE...

index.html is an empty stage with one marked spot; main.jsx is the stagehand who mounts the whole React play onto it.

Never ship code you cannot read

- Code that compiles and renders can still hide bugs the agent has no way to notice.
- Run every generated file past a short checklist before you trust it in your project.
- Common landmines: `key={index}`, a `useEffect` with no cleanup, a `fetch` with no error branch.
- Also check that no secret has been placed in a `VITE_` variable, where it becomes public.
- If you cannot explain why a line is there, you are not ready to ship it — that is the whole skill.

REMEMBER Audit generated code against a checklist before you trust it.

```
// Audit checklist before you run AI code:  
// - key={index} on a list?  
// - useEffect missing a cleanup return?  
// - fetch with no error state?  
// - a secret sitting in a VITE_ variable?  
// - SQL built by string concatenation?
```

THINK OF IT LIKE...

AI code that runs is a contract you have not read — it may work today and cost you everything on the clause you skipped.

A component is a function that returns JSX

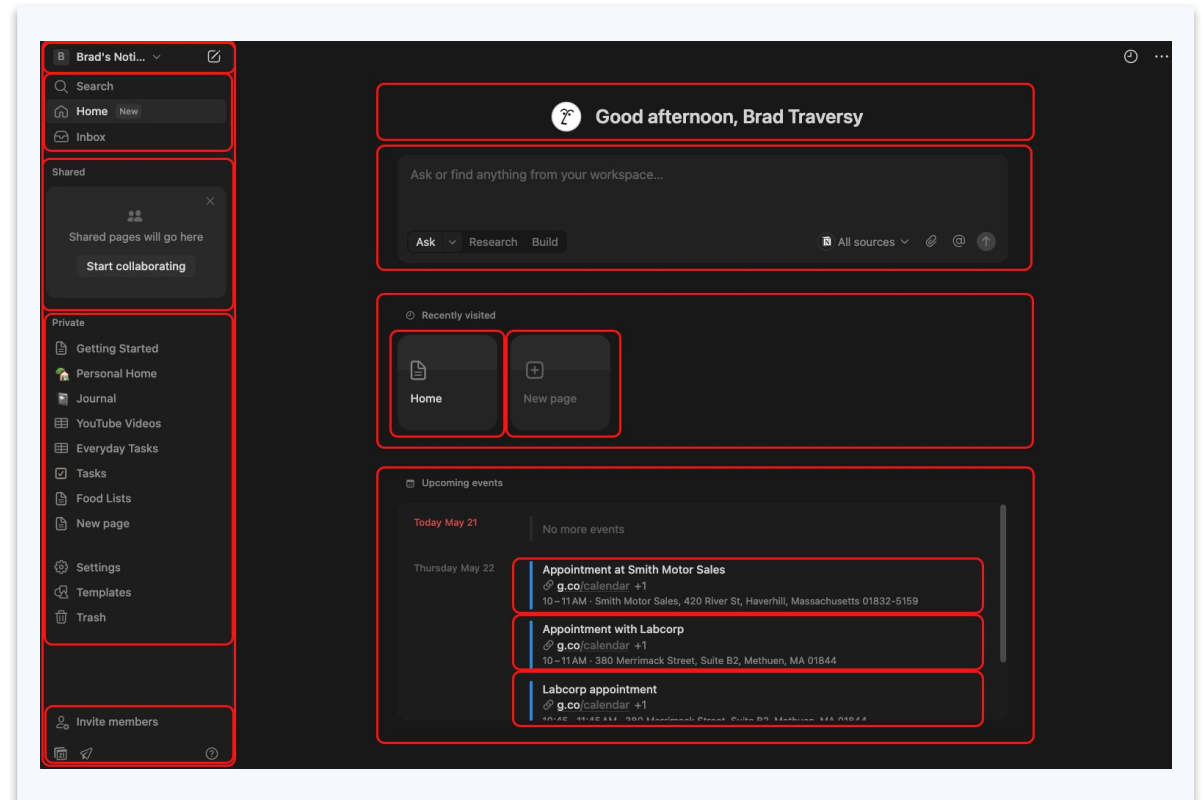
- This is the whole shape of a React component: a capitalised function that returns JSX.
- It destructures its props ({ name }) straight out of the parameter list, like function arguments.
- The {name} in the markup is a curly-brace window where a live JavaScript value flows in.
- export default makes the component importable and usable elsewhere as <WelcomeMessage />.

```
function WelcomeMessage({ name }) {  
  return <h1>Welcome, {name}!</h1>;  
}  
  
export default WelcomeMessage;
```

A minimal React component: props in through destructuring, JSX out, exported for reuse.

Everything on screen is a component

- Before you write a line of code, look at a real interface and draw boxes around its parts.
- Each box is a component: a sidebar, a page header, a row, a button, a comment.
- Boxes nest. A page contains a header; the header contains a title and an avatar.
- That nesting is exactly the component tree React renders, parent passing props to child.
- Naming the boxes first is how you decide what components to build.



A familiar app decomposed into boxes — this is the component tree, before any code exists.

Set Up Your Vibe Coding Workspace

LAB 1.1

The client wants a website for their cooking & bakery school. The learner installs Node, VS Code and an AI coding agent, scaffolds the cookbake Vite + React app, traces how index.html, main.jsx and App.jsx put React on the page, and runs the Prompt to Generate to Read to Understand to Correct loop for the first time.

You'll build

A running Vite + React dev server serving your own cookbake app at <http://localhost:5173>

Tech & files: Node 22, Vite, React 19, VS Code, Claude Code / Cursor / Copilot

Set Up Your Vibe Coding Workspace

Test it

The dev server runs at `http://localhost:5173` showing your Cook & Bake Academy heading, edits hot-reload instantly, and you can explain out loud what `index.html`, `main.jsx` and `App.jsx` each do.

Your First Vibe-Coded Component: CourseCard

LAB 1.2

The catalogue is 20 bakery and cooking courses, so the card that shows one course is the component the whole site is built from. The learner prompts the agent for CourseCard, then reads the output line by line: a component is a function, JSX is not HTML, and props are read-only arguments flowing one way from parent to child.

You'll build

A reusable `<CourseCard />` showing one course's emoji, title, level, fee, duration and campus

Tech & files: React function components, JSX, props, destructuring, `src/components/CourseCard.jsx`

CourseCard.jsx — your first component

FILE src/components/CourseCard.jsx

```
export default function CourseCard({
  title, category, level, fee,
}) {
  return (
    <article className="card">
      <span className="card__tag">{category}</span>
      <h3>{title}</h3>
      <span className="card__lvl">{level}</span>
      <strong>S${fee}</strong>
    </article>
  )
}
```

What this shows

- A component is a function whose name is Capitalised and returns JSX markup.
- It destructures its props — title, category, level, fee — from the single argument object.
- {title} is a curly-brace window where a live JavaScript value flows into the markup.

App.jsx — passing props in

FILE src/App.jsx

```
import CourseCard from './components/CourseCard'

export default function App() {
  return (
    <div className="container">
      <h1>Cook & Bake Academy</h1>
      <CourseCard
        title="Artisan Sourdough Bread Baking"
        category="Bakery"
        level="Beginner"
        fee={680}
      />
    </div>
  )
}
```

What this shows

- Props are passed like HTML attributes: strings in quotes, numbers in braces (fee={680}).
- Each <CourseCard .../> renders one card from the exact values you hand it.

Your First Vibe-Coded Component: CourseCard

Test it

One course card renders the sourdough course with its emoji, title, level, S\$680 fee, 4 weeks and campus; changing a prop in App.jsx changes what the card shows; and you can say why a component may never assign to its own props.

Vibe-Code the Cook & Bake Academy Landing Page

LAB 1.3

The learner vibe-codes the whole landing page — navbar, hero, a six-card course grid and a footer — from the client's mockup, then stops and audits it. The page works, but the AI left two smells behind: every component crammed into one file, and six copy-pasted cards. Naming them is the point of the lab.

You'll build

A full Cook & Bake Academy landing page: navbar, hero, six course cards and a footer

Tech & files: JSX composition, props, src/App.jsx, src/components/CourseCard.jsx, src/index.css

One file, six hand-typed cards

FILE src/App.jsx

```
function CourseGrid() {  
  return (  
    <section id="courses" className="container">  
      <h2>Popular courses</h2>  
      <div className="grid">  
        <CourseCard title="Artisan Sourdough Bread Baking"  
          category="Bakery" level="Beginner" fee={680} />  
        <CourseCard title="Macaron Masterclass"  
          category="Bakery" level="Intermediate" fee={420} />  
        { /* ...four more, all hand-typed... */ }  
      </div>  
    </section>  
  )  
}
```

What this shows

- Every card is the same component pasted again with different literal values.
- Notice the repetition — it is the smell Topic 3 removes with data plus `.map()`.

App composes small components

FILE src/App.jsx

```
export default function App() {  
  return (  
    <>  
      <Navbar />  
      <Hero />  
      <CourseGrid />  
      <Footer />  
    </>  
  )  
}
```

What this shows

- A component can render other components, nesting them like HTML tags.
- App reads like a table of contents: Navbar, Hero, CourseGrid, Footer.

Vibe-Code the Cook & Bake Academy Landing Page

Test it

The landing page renders the Cook & Bake navbar, hero, six real course cards and a footer; the grid is responsive; and you can name the two design flaws the AI left behind and say which later lab repairs each one.

END OF PHASE 01

Cook & Bake Academy can now show a real bakery site in the browser, built from your own components.

Every lab in this phase added to the SAME app — one project, growing, not a pile of throwaway demos.

PHASE 02

Put It on the Internet

Building Cook & Bake Academy — we add the exact same app live on a public URL, redeployed automatically on every git push

Put It on the Internet

THE STORY SO FAR

The bakery site runs only on your laptop.

WHAT WE BUILD NOW

We add the exact same app live on a public URL, redeployed automatically on every git push.

THE IDEAS THIS PHASE NEEDS

To build it, you will learn Git, the production build and environment variables.

Git is the vibe coder's safety net

- Commit before you ask the agent to change anything, so you always have a clean point to return to.
- `git diff` shows exactly which lines the agent touched, line by line, before you accept them.
- `git restore .` throws away the working changes and puts you back at your last commit.
- Small, frequent commits make it obvious which prompt introduced which change.
- Without this habit, one confident-but-wrong AI edit can quietly break code you cannot recover.

REMEMBER Commit before you prompt; diff to review; restore to undo.

```
git add -A && git commit -m 'before AI edit'
```

prompt the agent, then review:

```
git diff      # what did it change?
```

```
git restore .  # undo if it went wrong
```

THINK OF IT LIKE...

Git is a video-game save point: commit before you prompt, and any bad edit is one 'load' away from undone.

React compiles to static files

- `npm run build` bundles your source into a `dist/` folder of plain HTML, CSS and JavaScript.
- Babel has already compiled every piece of JSX away — the shipped bundle contains no JSX at all.
- Those files are static: no Node server runs them, so any file host on earth can serve them.
- `npm run preview` serves the built `dist/` locally so you can test the real production output.
- The API routes in `api/` are NOT part of this bundle — Vercel deploys them as separate functions.

REMEMBER A React build is a folder of static files any host can serve.

```
npm run build    # -> dist/ (html, css, js — no JSX)
npm run preview  # serve dist/ locally to test
```

```
# dist/assets/index-*.js is what every visitor
# downloads. Open it. Anything in there is public.
```

THINK OF IT LIKE...

`npm run build` is packing for a trip — your sprawling workshop becomes one sealed suitcase of plain files any host can carry.

Never commit secrets

- A .gitignore lists paths Git must never track, keeping junk and secrets out of your history.
- Always ignore node_modules (huge, reinstallable), dist (rebuildable), and .env.local (secret).
- A credential committed even once lives in the history forever, even after you delete the line.
- If a secret is ever pushed, treat it as leaked: rotate it immediately, do not just remove it.
- Commit .env.example instead — the variable NAMES, with placeholder values and no real secrets.

REMEMBER node_modules, dist and .env.local never belong in Git history.

```
# .gitignore
node_modules
dist
.env.local      # holds DATABASE_URL and JWT_SECRET

# .env.example  <- this one IS committed
DATABASE_URL=postgresql://USER:PASSWORD@...
JWT_SECRET=dev-secret-change-me
```

THINK OF IT LIKE...

A secret in Git history is a postcard, not a letter — once written on the back, everyone down the line can read it forever.

VITE_ variables are public. Always.

- Vite exposes only variables prefixed VITE_ to browser code, via `import.meta.env.VITE_SOMETHING`.
- It SUBSTITUTES the value straight into the .js file it ships. It is not looked up at runtime.
- So a VITE_ variable is visible to every visitor: open devtools, or just read `dist/assets/index-*.js`.
- A VITE_ variable may hold a public URL. It may never hold a password, a key or a token.
- A variable with no VITE_ prefix is never touched by Vite and never reaches the browser at all.

REMEMBER VITE_ vars are compiled into the public bundle — no secrets, ever.

```
# .env.local
VITE_SITE_NAME=Cook & Bake Academy # public. fine.
DATABASE_URL=postgresql://user:PASSWORD@... # SERVER ONLY
```

```
// Browser code can only ever see the first one:
const name = import.meta.env.VITE_SITE_NAME
```

THINK OF IT LIKE...

A VITE_ variable is printed on the flyer, not whispered — baked into the bundle for anyone to read.

Deploy on every push

- Connect your GitHub repo to Vercel once, and every push to main triggers a fresh build and deploy.
- This is continuous deployment: your live site always reflects the latest committed code.
- Every branch and pull request gets its own preview URL, so you can test before merging.
- Vercel also builds everything in `api/` into serverless functions, on the same domain as the site.
- Secrets like `DATABASE_URL` are set in the Vercel dashboard, encrypted — never in the repo.

REMEMBER Push to GitHub; Vercel rebuilds the site AND the API functions.

```
git push origin main
# Vercel rebuilds and deploys automatically:
# dist/ -> the static site
# api/ -> serverless functions
# Same domain, so the browser calls /api/courses
# with no CORS and no separate backend URL.
```

THINK OF IT LIKE...

Connecting Vercel to GitHub is a standing order to a baker — every push, a fresh loaf is baked and set out, no phone call needed.

SPA deep links need a rewrite

- A single-page app has only one real file, index.html; the router fakes the other paths in the browser.
- Type /courses directly or refresh, and the static host looks for that file on disk and 404s.
- The fix is a rewrite rule: send every path to /index.html and let React Router match it.
- But you must EXCLUDE /api/*, or your API calls get rewritten to index.html and return a web page.
- That is what "Unexpected token '<'" means: you fetched a URL and res.json() got HTML back.

REMEMBER Rewrite every path to /index.html — except /api/*, or the API breaks.

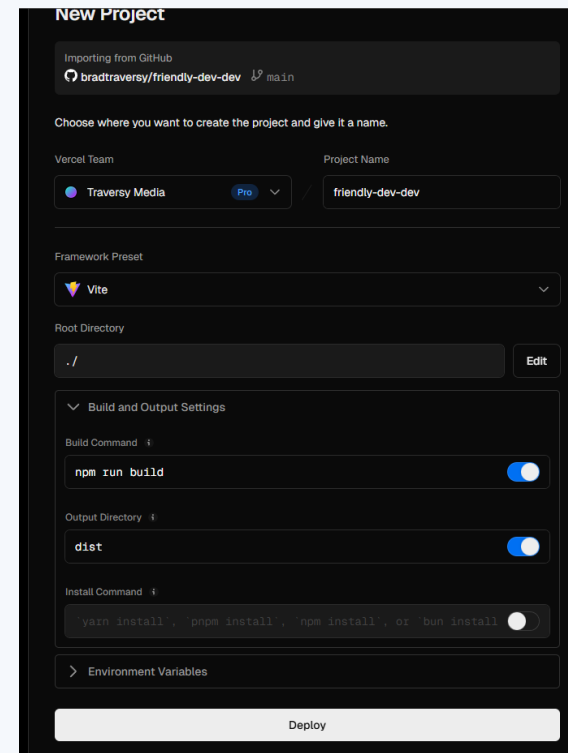
```
// vercel.json (real) — note the negative lookahead
{
  "rewrites": [
    { "source": "/((?!api/).*)", "destination": "/index.html" }
  ]
}
// Everything EXCEPT /api/* falls back to index.html.
```

THINK OF IT LIKE...

A static host is a receptionist with a filing cabinet — ask for /courses, find no such file, and you must be told to always hand back index.html.

Deploying from Git to the cloud

- You connect the host to your GitHub repository once; after that, a push is a deploy.
- The host clones your repo, runs `npm run build`, and serves the static `dist/` folder.
- Every branch gets its own preview URL, so you can review a change before it reaches users.
- Environment variables are set in the dashboard, not committed — and `VITE_` ones are public.
- Because they are baked in at build time, changing one requires a redeploy, not a restart.



Git-connected deployment: push to main, the host rebuilds and publishes automatically.

Version Control with Git and GitHub

LAB 2.1

The landing page only exists on one laptop, and the next lab hands it to an AI agent to rewrite. The learner initialises a repository, writes a `.gitignore` that keeps dependencies and secrets out of history, pushes cookbake to GitHub, and adopts the vibe coder's habit: commit before you prompt, diff after.

You'll build

A GitHub repository holding the Cook & Bake Academy app, with `node_modules`, `dist` and `.env.local` excluded

Tech & files: git, GitHub, gh CLI, `.gitignore`

Version Control with Git and GitHub

Test it

The cookbake repository exists on GitHub, git status is clean, node_modules and .env.local are untracked, and you can demonstrate reverting an AI edit with git diff followed by git restore.

Deploy Cook & Bake Academy to Vercel

LAB 2.2

The client wants to see their website. The learner inspects what `npm run build` actually emits, deploys the GitHub repo to Vercel so every push redeployes, adds the SPA rewrite that Topic 6's routing will need, and learns the `VITE_` rule: anything with that prefix is baked into the bundle and is therefore public.

You'll build

Cook & Bake Academy live on a public Vercel URL, redeployed automatically on every git push

Tech & files: Vite production build, Vercel, `vercel.json`, environment variables

Deploy Cook & Bake Academy to Vercel

Test it

The app is live on a public Vercel URL, a fresh git push triggers an automatic redeploy, the VITE_ value is visibly present inside dist/assets/*.js, and you can explain why a VITE_ variable can never hold a secret.

Deploy to GitHub Pages with GitHub Actions

LAB 2.3

Not every client uses Vercel. The learner vibe-codes a GitHub Actions workflow that builds and publishes the app on every push to main, then debugs the two classic static-host failures for themselves: a blank page caused by the wrong base path, and a 404 on refresh caused by the missing SPA fallback.

You'll build

A GitHub Actions workflow deploying Cook & Bake Academy to GitHub Pages on every push to main

Tech & files: GitHub Actions, actions/deploy-pages, vite.config.js base path

Deploy to GitHub Pages with GitHub Actions

Test it

The Actions workflow completes green, the site loads at <https://<user>.github.io/cookbake/> with its CSS and images intact, refreshing a sub-path does not 404, and you can explain what the base path and the 404.html fallback each fix.

END OF PHASE 02

Cook & Bake Academy can now send the client a link to their running website.

Every lab in this phase added to the SAME app — one project, growing, not a pile of throwaway demos.

Lunch Break

30 minutes

PHASE 03

Make It Real, Not Hardcoded

Building Cook & Bake Academy — we add real components driven by data, all 20 courses rendered with `map()`, and a working Bakery/Cooking filter and search

Make It Real, Not Hardcoded

THE STORY SO FAR

The page is copy-pasted markup — six course cards typed by hand.

WHAT WE BUILD NOW

We add real components driven by data, all 20 courses rendered with `map()`, and a working Bakery/Cooking filter and search.

THE IDEAS THIS PHASE NEEDS

To build it, you will learn the real DOM vs the Virtual DOM, how Babel compiles JSX, composition, lists and keys, and events.

The real DOM, and life without React

- The DOM is the browser's live object tree. Change a node's property and the screen changes.
- Without React you say HOW: `getElementById`, then overwrite `textContent`, `className`, `style`, `value`.
- The truth now lives in a DOM string, so you read '10 courses' back out and parse it to add to it.
- Every new feature multiplies the patches: filter the grid and you must also fix the count and the chip.
- One missed patch and the screen quietly disagrees with your data. Nothing throws. Nobody notices.

REMEMBER Hand-patching the DOM means saying HOW, in every place, every time.

```
// Vanilla: filtering the Cook & Bake catalogue.  
document.querySelectorAll('.card').forEach((card) => {  
  const show = card.dataset.category === 'Bakery'  
  card.style.display = show ? 'block' : 'none'  
})  
document.getElementById('count').textContent = '10 courses'  
// ...plus the chip, the heading, the empty state...
```

THINK OF IT LIKE...

The DOM is the page itself — a live object tree. Editing it by hand is rewiring a running machine.

The Virtual DOM and reconciliation

- React keeps a lightweight JavaScript tree — the Virtual DOM — describing what the UI should be.
- Building it is cheap: it is just plain objects. Touching the real DOM is the expensive part.
- On each render React builds a NEW virtual tree for the current state.
- Reconciliation is the diff: it compares the new tree to the previous one to find what changed.
- It patches only those differences into the real DOM. Unchanged nodes are never touched.

REMEMBER React diffs a virtual tree and patches only what actually changed.

```
// You describe the UI for this state:
```

```
<p>{visible.length} courses</p>
```

```
// Filter to Bakery: React builds a new virtual tree,
```

```
// diffs it, and patches ONE text node ('20' -> '10')
```

```
// plus removing 10 cards. The navbar, hero and footer
```

```
// were identical in both trees, so they are untouched.
```

THINK OF IT LIKE...

The Virtual DOM is an architect's blueprint — React compares the new drawing to the old before knocking down a single real wall.

Babel compiles JSX into function calls

- The browser has NEVER seen JSX. It is not JavaScript and no browser can parse it.
- Babel compiles it away at build time. Vite wires Babel in through `@vitejs/plugin-react`.
- `<CourseCard fee={680}/>` becomes `React.createElement(CourseCard, { fee: 680 })`.
- That call returns a plain JavaScript object — `{ type, props }` — not HTML and not a DOM node.
- A tree of those objects IS the Virtual DOM. This is the join between the last slide and this one.

REMEMBER Babel rewrites every JSX tag into a call that returns a plain object.

// YOU WRITE (JSX):

```
<CourseCard course={course} fee={680} />
```

// BABEL COMPILES IT TO (plain JavaScript):

```
React.createElement(CourseCard, { course: course, fee: 680 })
```

// WHICH RETURNS (a plain object — a React element):

```
{ type: CourseCard, props: { course: {...}, fee: 680 } }
```

THINK OF IT LIKE...

Babel is a translator in a booth — you speak JSX, and the browser only ever hears plain JavaScript.

Declarative beats imperative

- Imperative code lists the steps: find the element, set its text, toggle a class, and so on.
- Declarative code describes the end result for the current state and lets React reach it.
- In React you never write 'update this node' — you write what the UI is, as a function of state.
- This means the UI can never drift out of sync with your data the way manual DOM edits do.
- Change the state, and the correct UI follows automatically; that is React's core promise.

REMEMBER Describe the UI for the current state; React figures out the how.

```
// Imperative (vanilla): you do each step  
el.textContent = String(visible.length)
```

```
// Declarative (React): you state the result  
<p>{visible.length} courses</p>
```

THINK OF IT LIKE...

Declarative code is ordering a dish, not reciting the recipe — you state the result you want and let React cook.

State vs props

- Props are passed in from the parent and are read-only — the component cannot change them.
- State is owned by the component itself and can change over time in response to events.
- Changing state triggers a re-render; receiving new props from a parent also triggers a re-render.
- Ask 'does this component own this value, or is it given?' to decide between state and props.
- CourseCard is all props and no state. CoursesPage owns the query state and passes it down.

REMEMBER Props come from outside and are read-only; state is owned and changeable.

```
// searchBar owns nothing — pure props.
function SearchBar({ value, onChange }) {
  return <input value={value}
    onChange={(e) => onChange(e.target.value)} />
}

// CoursesPage owns the state:
const [query, setQuery] = useState("")
<SearchBar value={query} onChange={setQuery} />
```

THINK OF IT LIKE...

Props are the mail a component receives; state is the note it keeps on its own desk — one comes from outside, one it writes itself.

State: a component's memory

- State is data a component owns and remembers across renders, unlike a variable reset every render.
- A plain `let` is recomputed and lost each render, so changing it can never update what is on screen.
- `useState` gives a remembered value plus a setter: `const [query, setQuery] = useState("")`.
- Calling the setter does two things: it stores the new value and schedules a re-render to show it.
- That re-render is the point — the UI redraws from the new state, so screen and data stay in step.

REMEMBER State is remembered data; its setter stores the value and re-renders.

```
let q = "    // reset every render — useless
q = 'sushi'  // nothing on screen changes

const [q, setQ] = useState("")
setQ('sushi') // stored, AND schedules a re-render
```

THINK OF IT LIKE...

State is a component's memory — a notepad it keeps between renders, not scratch paper binned each time.

Composition and the children prop

- Composition means building complex UI by nesting simple components inside one another.
- The special children prop holds whatever JSX sits between a component's opening and closing tags.
- This lets Section, a Card or a Layout wrap arbitrary content without knowing what that content is.
- Favour composition over configuration — do not pile endless boolean props onto one component.
- The real Section takes id, title, eyebrow, alt and children — and renders anything you nest in it.

REMEMBER children lets a component wrap any content instead of endless props.

```
// src/components/Section.jsx (real code)
export default function Section({ title, children }) {
  return (
    <section className="section">
      {title} && <h2>{title}</h2>
      {children}
    </section>
  )
}

<Section title="Popular courses"><CourseGrid /></Section>
```

THINK OF IT LIKE...

The children prop is a picture frame — it does not care what photo you slide in, it just wraps whatever you hand it.

Rendering lists with map()

- You render a list by calling map() to turn an array of data into an array of JSX elements.
- React accepts an array of elements directly inside JSX and renders them in order.
- This is what kills the hand-typed cards from Topic 1: 20 courses, one CourseGrid, one map().
- Adding a course is now a data edit, not a markup edit — and the grid updates itself.
- Every element in the returned list needs a key, which the next slide explains.

REMEMBER map() turns a data array into an array of elements React renders.

```
// src/components/CourseGrid.jsx (real code)
<div className="grid">
  {courses.map((course) => (
    <CourseCard key={course.id} course={course} />
  ))}
</div>
```

THINK OF IT LIKE...

map is a name-tag printer: feed it a list of students and it hands back one printed tag per person, in order.

key is identity, not decoration

- A key is a stable id React uses to track which item is which across renders.
- It lets React move, keep or remove the right cards instead of rebuilding the whole grid.
- Use a stable id from your data (course.id), never the array index, as the key.
- `key={index}` looks fine until you filter, reorder or delete — then state attaches to the wrong card.
- This is one of the single most common bugs in AI-generated list code, so always check it.

REMEMBER Key each card by a stable id; `key={index}` corrupts state on change.

```
{courses.map((course) => (  
  <CourseCard key={course.id} course={course} />  
))}
```

```
// key={index} corrupts state when you filter to Bakery:  
// the card at index 0 is now a different course, but  
// React thinks it is the same one and keeps its state.
```

THINK OF IT LIKE...

A key is a student's name tag, not their seat number — reshuffle the room and React still knows exactly who is who.

Conditional rendering and the 0 && trap

- Use a ternary in JSX to choose between two elements based on a condition.
- Use && to render an element only when a condition is true, and nothing otherwise.
- The footgun: {seats && <Badge/>} renders a literal 0 on screen when seats is 0, because 0 is falsy.
- Fix it by making the left side a real boolean: {seats > 0 && <Badge/>}.
- This is subtle and the agent produces it often, so scan every && for a number on its left.

REMEMBER Gate && on a real boolean (seats > 0), never a bare number.

```
// From CourseCard: a ternary picks one of two.  
<span>{isBakery ? '🍰 Bakery' : '👨🍳 Cooking'}</span>
```

```
// && renders one thing or nothing:  
{seats > 0 && <span>{seats} places left</span>}
```

```
// {seats && ...} prints a literal 0 when seats is 0.
```

THINK OF IT LIKE...

0 && is a trapdoor — zero is falsy but not nothing, so React renders a bare 0 unless you gate on seats > 0.

Derived state must not be stored

- If a value can be computed from existing state during render, compute it. Do not store it.
- The filtered course list is derived from courses + category + query. It is not its own state.
- Mirroring it into useState creates two sources of truth, and they will drift apart.
- Computing on render is cheap and always correct; a stored copy needs syncing you will forget.
- The agent loves to add a useState for filtered lists and totals. Delete it and compute inline.

REMEMBER Compute derived values on render; never mirror them into useState.

```
// src/pages/CoursesPage.jsx (real code) — DERIVED.  
const visible = courses.filter((c) => {  
  const matchesCat = category === 'All'  
    || c.category === category  
  const matchesQ = !needle  
    || c.title.toLowerCase().includes(needle)  
  return matchesCat && matchesQ  
})
```

THINK OF IT LIKE...

Storing a derived value is photocopying a number already on the page — keep the copy and it soon disagrees with the original.

Synthetic events

- You attach handlers with camelCase props like `onClick` and `onChange`, passing a function.
- Pass the function, do not call it: `onClick={handleEnroll}`, not `onClick={handleEnroll()}`.
- To pass an argument, wrap it in an arrow: `onClick={() => onChange(cat.value)}`.
- React hands your handler a synthetic event, a cross-browser wrapper over the native event.
- Read the input's value from `e.target.value`; call `e.preventDefault()` to stop a form reloading.

REMEMBER Pass a function to `onClick`; wrap in an arrow to pass an argument.

```
// src/components/CategoryFilter.jsx (real code)
{categories.map((cat) => (
  <button key={cat.value}
    onClick={() => onChange(cat.value)}>
    {cat.label}
  </button>
))}
// onClick={onChange(cat.value)} would CALL it on render.
```

THINK OF IT LIKE...

A synthetic event is a universal remote — React wraps every browser's quirks in one consistent set of buttons.

Controlled inputs

- In a controlled input, React state is the single source of truth for the field's value.
- You bind `value={state}` and update it in `onChange`, so state and the input never disagree.
- This unlocks live search, validation, formatting and disabling a submit button as the user types.
- A value with no `onChange` makes the field read-only — React warns you in the console.
- `SearchBar` is controlled and owns nothing: the value comes down, every keystroke goes back up.

REMEMBER Bind value to state and update it in `onChange` — React owns the field.

```
// src/components/SearchBar.jsx (real code)

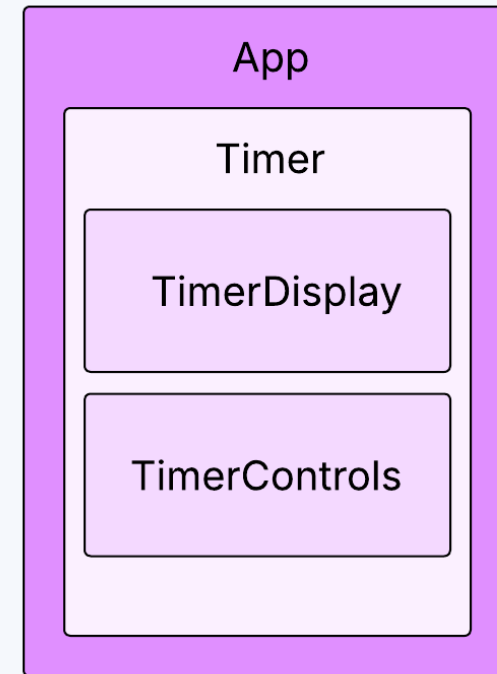
```

THINK OF IT LIKE...

A controlled input is a puppet on React's strings — state moves the field, and every keystroke reports back to pull the string.

An app is a tree of nested components

- A React app is a tree: App contains Timer, which contains TimerDisplay and TimerControls.
- Each box is one component with one job, composed together to build the whole interface.
- Data flows down this tree as props; a parent passes values to the children it renders.
- Breaking UI into small nested components is what makes it readable, reusable and testable.



Composition in practice: a parent component nests smaller, single-purpose children.

The Real DOM, the Virtual DOM and Babel

LAB 3.1

Before improving the app, the learner finds out what React is doing on their behalf. They build the Cook & Bake course grid TWICE: once by hand with `document.createElement` and `innerHTML` (the real DOM), and once in React. They then see, with their own eyes, that the browser never receives a single line of JSX — Babel has already compiled it into `React.createElement` calls that return plain JavaScript objects, and it is those objects that

You'll build

A hand-built vanilla DOM grid at `/vanilla.html`, sitting next to the React grid — and the compiled output of your own JSX

Tech & files: `document.createElement`, `innerHTML`, the Virtual DOM, reconciliation, Babel, `@vitejs/plugin-react`, `babeljs.io/repl`

Without React: you patch the DOM by hand

FILE (vanilla JS, for contrast)

```
function filterBakery() {  
  // Find every card and toggle it, one by one.  
  document.querySelectorAll('.card').forEach((card) => {  
    const show = card.dataset.category === 'Bakery'  
    card.style.display = show ? 'block' : 'none'  
  })  
  // ...and now keep the count in step, by hand:  
  const el = document.getElementById('count')  
  el.textContent = '10 courses'  
}
```

What this shows

- The vanilla-JS way: find nodes and mutate them — you own every step of HOW.
- The truth now lives in a DOM string; reading '10' back out to add to it is your problem.
- Every extra feature is another patch you must remember to keep in sync.

With React: you describe the result

FILE src/pages/CoursesPage.jsx

```
const [category, setCategory] = useState('All')

const visible = courses.filter(
  (c) => category === 'All' || c.category === category
)

return (
  <>
    <p>{visible.length} courses</p>
    <CourseGrid courses={visible} />
  </>
)
```

What this shows

- State holds the filter; you only declare WHAT the UI is for the current value.
- setCategory re-renders — React builds a new virtual tree, diffs it, and patches only what changed.
- No getElementById, no textContent, no style: the count and the grid can never drift apart.

The Real DOM, the Virtual DOM and Babel

Test it

Both grids show the same six courses, but you can demonstrate that the vanilla one rebuilds every card on each keystroke while React repaints only what changed; you can show the Babel output of your own JSX as a `React.createElement` call; and you can state what the Virtual DOM object tree is and where the ONE real-DOM write in a React app lives (`main.jsx`).

Components, Props and Composition

LAB 3.2

This lab pays off smell 1 from Lab 1.3. The learner moves the Cook & Bake catalogue into `src/data/courses.js`, splits `App.jsx` into real component files, builds a `Section` layout component that wraps arbitrary content with the `children` prop, and refactors `CourseCard` to take ONE course object — the same shape the Neon Postgres row will have in Topic 5, so the card will never need changing again.

You'll build

Navbar, Hero, Section, CourseGrid, CourseCard and Footer as separate files, composed by `App.jsx` and driven by `src/data/courses.js`

Tech & files: Components, props, the `children` prop, composition, `src/data/courses.js`, `src/components/*.jsx`

CourseCard takes one course object

FILE src/components/CourseCard.jsx

```
// One prop, one object — the shape of a DB row.
export default function CourseCard({ course }) {
  const { slug, title, category, level, fee } = course
  const isBakery = category === 'Bakery'
  return (
    <Link to={` /courses/${slug}`} className="card">
      <span className="card__tag">
        {isBakery ? '🍰 Bakery' : '🍳 Cooking'}
      </span>
      <h3>{title}</h3>
      <span className="card__price">S${fee}</span>
    </Link>
  )
}
```

What this shows

- One `course` prop replaces five — its shape matches the DB row you fetch in Topic 5.
- Destructuring pulls the fields you need out of the object in a single line.
- The whole card is a <Link>, so a click routes to the detail page with no reload.

Section composes with children

FILE src/components/Section.jsx

```
export default function Section({ title, children }) {  
  return (  
    <section className="section">  
      {title} && <h2>{title}</h2>  
      {children}  
    </section>  
  )  
}
```

```
// Used as a wrapper around anything:  
<Section title="Popular courses">  
  <CourseGrid courses={popular} />  
</Section>
```

What this shows

- Whatever you nest between the tags arrives as the children prop.
- Section wraps CourseGrid without knowing what it is — composition over configuration.

Components, Props and Composition

Test it

App.jsx defines no component other than App, the page renders exactly as before, CourseCard takes a single `course` prop, and adding a 21st course to src/data/courses.js requires no change to any component.

Lists, Keys and Conditional Rendering

LAB 3.3

This lab pays off smell 2 from Lab 1.3. Six copy-pasted cards become one `.map()` over all 20 courses. The learner then deliberately BREAKS it with `key={index}` to see state stick to the wrong course card, and meets the 0 && footgun that puts a stray zero on the page.

You'll build

A CourseGrid rendering all 20 Cook & Bake courses from data, with a level badge and a real empty state

Tech & files: `Array.map`, the `key` prop, reconciliation, conditional rendering, `src/components/CourseGrid.jsx`

Render a list with .map() and key

FILE src/components/CourseGrid.jsx

```
export default function CourseGrid({ courses }) {  
  if (!courses.length) {  
    return <p className="muted">No courses match your search.</p>  
  }  
  return (  
    <div className="grid">  
      {courses.map((course) => (  
        <CourseCard  
          key={course.id}  
          course={course}  
        />  
      ))}  
    </div>  
  )  
}
```

What this shows

- `courses.map()` turns an array of data into an array of `<CourseCard>` elements.
- `key={course.id}` gives each card a stable identity so React reconciles efficiently.
- Use the data's own id as the key — never the array index.
- An early return handles the empty list first, so the JSX below can assume data.

Conditional rendering & the 0 trap

FILE src/components/CourseCard.jsx

```
// Ternary: choose one of two things to render.
<span className="card__tag">
  {isBakery ? '🍰 Bakery' : '👨🍳 Cooking'}
</span>

<span>🕒 {weeks} week{weeks > 1 ? 's' : ''}</span>

{/* Footgun: `seats > 0 &&`, never `seats &&`.
  A bare 0 renders a literal "0" on the card. */}
{seats > 0 && <p>{seats} places left</p>}
```

What this shows

- A ternary picks one of two elements: the Bakery tag vs the Cooking tag.
- `{cond && <x/>}` renders x only when cond is true.
- Trap: write `seats > 0 &&`, not `seats &&` — a bare 0 renders a literal '0' on screen.

Lists, Keys and Conditional Rendering

Test it

All 20 courses render from `src/data/courses.js` with `key={course.id}`, the home page shows 6, Advanced courses show the badge, an empty array shows the empty state, no literal 0 appears anywhere, and you can demonstrate the `key={index}` bug and explain it.

Events, the Bakery / Cooking Filter and a Controlled Search Box

LAB 3.4

The final piece of the phase: the mockup's All / 🍰 Bakery / 🔍 Cooking chips and a live search box. The learner meets React's synthetic events and the controlled input, and then the single most common mistake in AI-generated React — mirroring a value that can simply be COMPUTED into a second piece of state.

You'll build

A working toolbar over the catalogue: a SearchBar and the CategoryFilter chips, filtering all 20 courses live

Tech & files: Synthetic events, onClick, onChange, onSubmit, controlled inputs, useState (preview), src/components/{SearchBar,CategoryFilter}.jsx

A controlled input

FILE src/components/SearchBar.jsx

```
export default function SearchBar({ value, onChange }) {  
  return (  
    <input  
      type="search"  
      value={value}  
      onChange={(e) => onChange(e.target.value)}  
      placeholder="Search sourdough, sushi, macaron..."  
    />  
  )  
}
```

What this shows

- value comes from state and onChange reports every keystroke up to the parent.
- React state is the single source of truth — the DOM input only displays it.

Derived data, not extra state

FILE src/pages/CoursesPage.jsx

```
const category = searchParams.get('category') ?? 'All'
const needle = q.toLowerCase()

// Derived data — NOT state. Recomputed each render.
const visible = courses.filter((c) => {
  const matchesCat =
    category === 'All' || c.category === category
  const matchesQ =
    !needle ||
    c.title.toLowerCase().includes(needle) ||
    c.code.toLowerCase().includes(needle)
  return matchesCat && matchesQ
})
```

What this shows

- `visible` is recomputed from courses, category and the query on every render.
- If you can calculate it, don't store it — duplicating it in state causes stale lists.

Passing an argument to a handler

FILE src/components/CategoryFilter.jsx

```
{categories.map((cat) => (  
  <button  
    key={cat.value}  
    className={  
      cat.value === value ? 'chip is-active' : 'chip'  
    }  
    onClick={() => onChange(cat.value)}  
  >  
    {cat.label}  
  </button>  
))}
```

What this shows

- Wrap the call in an arrow: `onClick={() => onChange(cat.value)}`.
- `onClick={onChange(cat.value)}` would CALL it during every render — a bug.

Events, the Bakery / Cooking Filter and a Controlled Search Box

Test it

Typing in the search box filters the 20 courses live, the 🍰 Bakery and 🔍 Cooking chips narrow the grid further and combine with the search, an unmatched search shows the empty state, and the filtered array exists only as a local variable — never in `useState`.

END OF PHASE 03

Cook & Bake Academy can now add a course by editing data, not markup — and filter the catalogue as you type.

Every lab in this phase added to the SAME app — one project, growing, not a pile of throwaway demos.

End of Day 1

See you tomorrow at 9:30am

PHASE 04

Make It Remember and React

Building Cook & Bake Academy — we add a shortlist of courses, a dark-mode theme, a search box that focuses itself, and reusable logic — all powered by hooks

Make It Remember and React

THE STORY SO FAR

A catalogue you can only look at — it forgets everything on refresh.

WHAT WE BUILD NOW

We add a shortlist of courses, a dark-mode theme, a search box that focuses itself, and reusable logic — all powered by hooks.

THE IDEAS THIS PHASE NEEDS

To build it, you will learn the hooks: `useState`, `useEffect`, `useRef` (the DOM escape hatch), `useContext`, `useReducer` and custom hooks.

The Rules of Hooks

- Call hooks only at the top level of a component or hook — never in a loop, condition or nested function.
- React identifies each hook purely by its call order, which must be identical on every render.
- A hook inside an if runs some renders and not others, and the whole list of hooks shifts and breaks.
- Only call hooks from React function components or from your own custom hooks.
- The lint rules for hooks catch most violations — do not disable them to silence a warning.

REMEMBER Call hooks at the top level, in the same order, every render.

```
function CoursesPage({ open }) {  
  const [q, setQ] = useState("") // ok: top level  
  if (open) {  
    const [x] = useState(1)      // never: in an if  
  }  
}
```

THINK OF IT LIKE...

Hooks are called in roll-call order — React counts 'hook one, hook two' every render, so you must never skip or reorder a name.

What a hook actually is

- A hook is a function whose name starts with 'use' that lets a function component use React features.
- Hooks let a stateless function remember values (useState) and sync with the outside (useEffect).
- React identifies each hook by call order, matching it to a memory slot by position, not by name.
- That is why hooks live at the top level — never inside an if, a loop, or after an early return.
- Only components and other hooks may call hooks; a plain helper function cannot.

REMEMBER A hook is a use...() function; React tracks it by call order.

```
function CoursesPage() {  
  const { courses } = useCourses() // custom hook  
  const [q, setQ] = useState("") // slot 2  
  const searchInput = useRef(null) // slot 3  
  useEffect(() => { // slot 4  
    searchInput.current?.focus()  
  }, [])  
}
```

THINK OF IT LIKE...

A hook is a power outlet on the render: plug in with use...() and your function taps React's memory and lifecycle.

State is a snapshot

- useState returns the current value and a setter: `const [count, setCount] = useState(0)`.
- Within one render the state value never changes — it is a snapshot captured at render time.
- Calling `setCount(count + 1)` twice in a row adds only one, because both read the same stale count.
- When the next value depends on the previous, use the updater form: `setCount(c => c + 1)`.
- Setting state schedules a re-render; it does not change the current variable mid-function.

REMEMBER State is fixed per render; use `setX(prev => ...)` for dependent updates.

```
setCount(count + 1) // stale: +1 total  
setCount(count + 1)
```

```
setCount((c) => c + 1) // updater: +1  
setCount((c) => c + 1) // updater: +2
```

THINK OF IT LIKE...

State is a photograph, not a live feed — count is frozen at the value it had when this render's photo was taken.

Immutable updates

- React decides to re-render by checking whether the state value is a NEW reference.
- `items.push(course)` mutates the same array, so the reference is unchanged and nothing re-renders.
- Build a new value with spread instead: `setItems([...items, course])` creates a fresh array.
- Update an object the same way: `setCourse({ ...course, fee: 720 })`.
- The real `CartContext` never mutates: every add, remove and clear returns a brand-new array.

REMEMBER Never mutate state — build a new array or object with spread.

```
// src/context/CartContext.jsx (real code)
const addItem = (course) =>
  setItems((prev) =>
    prev.some((c) => c.id === course.id)
      ? prev
      : [...prev, course]) // NEW array -> re-renders

const removeItem = (id) =>
  setItems((prev) => prev.filter((c) => c.id !== id))
```

THINK OF IT LIKE...

Mutating state is editing the original photo — React only re-renders when you hand it a brand-new print.

useEffect and the dependency array

- useEffect runs code after render for side effects: fetching, subscriptions, timers, the DOM.
- Its second argument, the dependency array, decides when it re-runs.
- An empty array [] runs the effect once after the first render, and never again.
- Listing [slug] re-runs the effect whenever slug changes — that is how a new course gets fetched.
- Omit the array entirely and it runs after every render — the classic infinite-fetch loop.

REMEMBER [] runs once; [dep] re-runs when dep changes; no array runs always.

```
// src/context/ThemeContext.jsx (real code)
useEffect(() => {
  document.documentElement.dataset.theme = theme
}, [theme]) // re-runs only when the theme changes
```

THINK OF IT LIKE...

The dependency array is a guest list — the effect only re-runs when a name on the list actually changes.

Every effect cleans up after itself

- An effect that starts something ongoing must stop it, or you leak timers, listeners and requests.
- Return a cleanup function from the effect; React runs it before the next effect and on unmount.
- `clearTimeout`, `clearInterval`, `removeEventListener` and `unsubscribe` all belong in that function.
- In development, `StrictMode` mounts, unmounts and remounts once to expose effects with no cleanup.
- So a double log in dev is not a bug — it is React helping you find a missing cleanup.

REMEMBER Return a cleanup for anything ongoing; `StrictMode` double-runs to test it.

```
// src/hooks/useDebounce.js (real code)
useEffect(() => {
  const id = setTimeout(() => setDebounced(value), delay)
  return () => clearTimeout(id) // cleanup
}, [value, delay])
// Every keystroke cancels the previous pending timer,
// so only the last one ever fires.
```

THINK OF IT LIKE...

`useEffect` cleanup is turning the tap off as you leave the room — `StrictMode` runs in and out twice to catch a tap left running.

useRef: the escape hatch to the DOM

- useRef returns a mutable object, { current }, whose value persists across renders like state does.
- But unlike state, writing to .current never triggers a re-render — that is its defining difference.
- Job one: a live handle on a real DOM node, for the things JSX cannot describe.
- Job two: a mutable box for a value that must survive renders but must not drive the UI.
- In React 19 ref is a normal prop, so you pass it straight to your own components — no forwardRef.

REMEMBER Two jobs — a DOM handle and a mutable box; neither triggers a re-render.

```
const searchInput = useRef(null) // job one: a DOM handle
const timer = useRef(null)      // job two: a mutable box

// .current is null on the first render — React has not
// created the node yet. It is set right after paint.
useEffect(() => {
  searchInput.current?.focus()
}, [])

<input ref={searchInput} /> // React 19: ref is a prop
```

THINK OF IT LIKE...

A ref is a sticky note the renderer never reads — write to .current all you like and React will not repaint the screen.

ref vs state — the dividing line

- The whole decision is one question: should changing this value REDRAW the screen?
- If yes, it is STATE. The search text is state — every keystroke must repaint the results.
- If no, it is a REF. Focus, scroll position and a timer id change nothing that is rendered.
- Put a timer id in state and you re-render the entire component on every single tick. For nothing.
- 'Is focused' is browser state, not React state. There is no JSX you can write to express it.

REMEMBER Redraw the screen? State. Must not redraw? Ref. That is the whole rule.

```
// STATE — the screen must change:  
const [query, setQuery] = useState("")  
  
// REF — the screen must NOT change:  
const searchInput = useRef(null) // which DOM node  
const timer = useRef(null)      // the interval id  
  
// timer in useState => a re-render every tick, forever.
```

THINK OF IT LIKE...

State is what the audience sees; a ref is the stagehand's clipboard — necessary, remembered, and never on stage.

Three real refs in Cook & Bake

- CoursesPage focuses the search box on mount, and again after every category chip click.
- The ref is created in the PARENT and handed down to SearchBar as an ordinary prop.
- HomePage points a ref at the course grid, so the hero's 'Browse courses' button can scroll to it.
- Hero receives onBrowse as a function prop — it never knows a ref or a DOM node exists.
- A timer id lives in a ref because clearing it later must not cost the user a re-render.

REMEMBER Focus, scroll and timer ids: the sanctioned reasons to hold a DOM node.

```
// CoursesPage.jsx — focus the search input
const searchInput = useRef(null)
useEffect(() => { searchInput.current?.focus() }, [])
<SearchBar inputRef={searchInput} ... />

// HomePage.jsx — scroll the grid into view
const popularRef = useRef(null)
const scrollToCourses = () =>
  popularRef.current?.scrollIntoView({ behavior: 'smooth' })
<Hero onBrowse={scrollToCourses} />
<section ref={popularRef}>...</section>
```

THINK OF IT LIKE...

Refs are the three things the declarative world cannot say: put the cursor here, scroll there, hold this receipt.

useContext ends prop drilling

- Prop drilling is threading a value through components that do not use it, just to reach a deep child.
- Context provides a value once high in the tree and lets any component below read it directly.
- Create a context, wrap a subtree in its Provider, and read it with useContext — or a custom hook.
- Cook & Bake has three: ThemeContext (dark mode), AuthContext (the user), CartContext (shortlist).
- Do not use context for fast-changing values like keystrokes — every consumer re-renders on change.

REMEMBER Provide once, read anywhere with useContext — no more prop drilling.

```
// src/context/CartContext.jsx (real code)
export function useCart() {
  const ctx = useContext(CartContext)
  if (!ctx) throw new Error('useCart needs <CartProvider>')
  return ctx
}

// Anywhere, at any depth, with no prop drilling:
const { count } = useCart()
```

THINK OF IT LIKE...

Context is a building's PA system — announce once and any floor hears it, instead of passing a note desk to desk down every level.

useReducer and custom hooks

- useReducer suits state with several related actions, like a shortlist's add, remove and clear.
- A reducer is a pure function (state, action) => newState — same inputs always give the same output.
- It must build and return new state, never mutate the state it was handed.
- A custom hook is a function starting with 'use' that bundles hook logic for reuse.
- Custom hooks share LOGIC, not state — each caller of useLocalStorage gets its own key and value.

REMEMBER Reducers are pure (state, action) => newState; custom hooks share logic, not state.

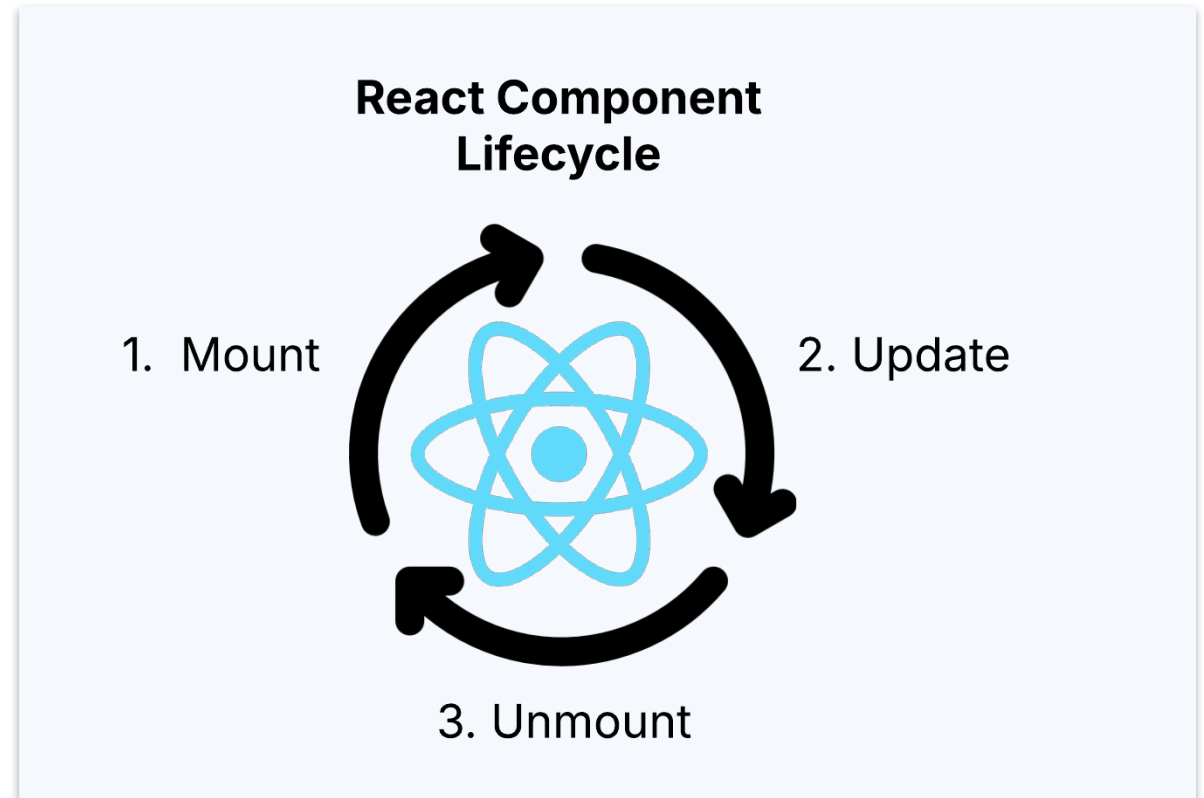
```
function cartReducer(state, action) {  
  switch (action.type) {  
    case 'ADD':  
      return state.some((c) => c.id === action.course.id)  
        ? state  
        : [...state, action.course]  
    case 'REMOVE':  
      return state.filter((c) => c.id !== action.id)  
    default: return state  
  }  
}
```

THINK OF IT LIKE...

A reducer is a vending machine — press a labelled button (an action) and get a predictable new state, every time.

Every component has a lifecycle

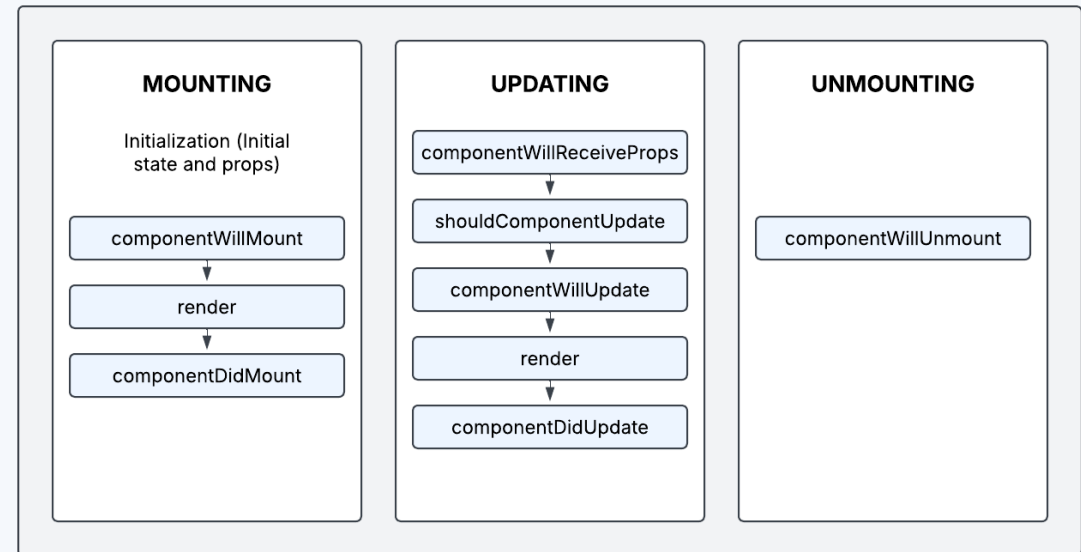
- Every component passes through three phases: it mounts, it updates, and it unmounts.
- Mount is the first render; update happens whenever state or props change; unmount is removal.
- With function components you tap into all three phases through the `useEffect` hook.
- This cycle is why effects need cleanup — the unmount phase is where you tear things down.



Mount, update, unmount — the three phases `useEffect` lets you hook into.

The class lifecycle useEffect replaced

- This is the OLD class-component API. You will not write any of these methods in this course.
- Logic for one feature was scattered across `componentDidMount`, `componentDidUpdate` and `componentWillUnmount`.
- Subscribe in one method, update in a second, unsubscribe in a third — easy to forget the third.
- One `useEffect` replaces all three: the body handles mount and update, the returned function handles unmount.
- You will still meet this diagram in older codebases and in AI-generated code trained on them.



The class-era lifecycle methods. A single `useEffect` now covers all three columns.

useState — State, Snapshots and Immutability

LAB 4.1

Cook & Bake Academy can be browsed but not USED. The learner adds a course shortlist owned by App, meets the Rules of Hooks, and learns why state is a snapshot, why the updater form exists, and why mutating an array never re-renders.

You'll build

A course shortlist owned by App, with Add / Remove buttons on every course card and a running S\$ total

Tech & files: useState, the Rules of Hooks, immutable updates, lifting state up

useState & immutable updates

FILE src/context/CartContext.jsx

```
// The shortlist lives in context, persisted to storage.
const [items, setItems] = useState('cart', [])

const addItem = (course) =>
  setItems((prev) =>
    // Immutable: build a NEW array, never push().
    prev.some((c) => c.id === course.id)
      ? prev
      : [...prev, course])

const removeItem = (id) =>
  setItems((prev) => prev.filter((c) => c.id !== id))
```

What this shows

- The shortlist lives once, in CartContext — the single owner of that state.
- Build a NEW array with the spread ...prev; never mutate with items.push().
- The updater form (prev => ...) always reads the latest state, so it is safe.

A 'dumb' presentational component

FILE src/components/CourseCard.jsx

```
// CourseCard holds no cart state. It just renders a
// course and links to it. The Add-to-shortlist button
// lives on the detail page and calls addItem(course)
// from context — the card stays pure.
export default function CourseCard({ course }) {
  const { slug, title, fee } = course
  return (
    <Link to={` /courses/${slug}`} className="card">
      <h3>{title}</h3>
      <span className="card__price">S${fee}</span>
    </Link>
  )
}
```

What this shows

- CourseCard holds no state; everything it draws arrives in the single `course` prop.
- State lives in context, read where it is needed — the card is reusable anywhere.

useState — State, Snapshots and Immutability

Test it

Clicking 'Add to shortlist' on Artisan Sourdough (S\$680) adds it and the button flips to Remove; the navbar count and the S\$ total both update; adding the same course twice does nothing.

useEffect — Side Effects and Cleanup

LAB 4.2

The shortlist is wiped on every refresh. The learner writes three effects — a document title, a localStorage save, and an enrolment-deadline countdown — and learns the dependency array, the cleanup function, and why StrictMode runs effects twice.

You'll build

A live browser-tab title, a shortlist persisted to localStorage, and a countdown to the next intake

Tech & files: useEffect, dependency arrays, cleanup functions, StrictMode, localStorage

useEffect with a cleanup

FILE src/hooks/useDebounce.js

```
export function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value)

  useEffect(() => {
    const id = setTimeout(
      () => setDebounced(value), delay
    )
    // Cleanup cancels the pending timer on every
    // keystroke — without it, a fast typist fires
    // the effect early and the search feels janky.
    return () => clearTimeout(id)
  }, [value, delay])

  return debounced
}
```

What this shows

- `setTimeout` schedules the update; the returned function is the cleanup.
- Each keystroke's cleanup clears the previous timer, so only the last one fires.
- `StrictMode` double-invokes this in dev to prove the cleanup is really there.

An effect that syncs with state

FILE src/context/ThemeContext.jsx

```
const [theme, setTheme] = useLocalStorage('theme', 'light')

// The one side effect: mirror the chosen theme onto
// <html data-theme="...">, which index.css keys its
// dark-mode variables off. Re-runs only when theme changes.
useEffect(() => {
  document.documentElement.dataset.theme = theme
}, [theme])

const toggle = () =>
  setTheme((t) => (t === 'dark' ? 'light' : 'dark'))
```

What this shows

- The effect re-runs only when `theme` changes — that is what [theme] declares.
- Touching document is a side effect, so it belongs in an effect, never in render.
- useLocalStorage persists the choice, so the theme survives a refresh.

useEffect — Side Effects and Cleanup

Test it

The tab title tracks the shortlist, the shortlist survives a page refresh, the intake countdown ticks exactly once per second, and removing the `clearInterval` visibly doubles its speed.

useRef — The DOM Escape Hatch, and What Is NOT State

LAB 4.3

Two things the Cook & Bake catalogue needs are impossible to describe in JSX: putting the cursor in the search box, and scrolling the page to the course grid. Both are browser state, not UI description. The learner uses useRef for both, then draws the dividing line between a ref and state and proves it with a timer.

You'll build

A search box that focuses itself on mount and after every filter chip (CoursesPage), and a hero 'Browse courses' button that smooth-scrolls to the grid (HomePage)

Tech & files: useRef, DOM refs, ref as an ordinary prop in React 19, scrollToView, focus

ref vs state — the dividing line

FILE src/pages/CoursesPage.jsx

```
// STATE: the search text. Changing it MUST redraw the
// results, so it is state.
const [query, setQuery] = useState("")

// REF: the real <input> DOM node. Focusing it changes
// nothing that is rendered, so it is a ref — bumping
// .current never triggers a re-render.
const searchInput = useRef(null)

// A timer id would be a ref too: clearing it later
// must not cost the user a re-render.
```

What this shows

- The whole decision: should changing it redraw the screen? Yes = state, no = ref.
- 'Is focused' is browser state — there is no JSX that can express it, so you need a ref.
- A timer id in useState would re-render the component on every tick, for nothing.

useRef for focus & scroll

FILE src/pages/CoursesPage.jsx · HomePage.jsx

```
// CoursesPage: focus the search box on mount, and
// again after every category chip click.
const searchInput = useRef(null)
useEffect(() => { searchInput.current?.focus() }, [])
<SearchBar value={query} onChange={setQuery}
  inputRef={searchInput} />

// HomePage: the hero button scrolls to the grid.
const popularRef = useRef(null)
const scrollToCourses = () =>
  popularRef.current?.scrollIntoView({ behavior: 'smooth' })
<Hero onBrowse={scrollToCourses} />
<section ref={popularRef}>...</section>
```

What this shows

- `useRef(null)` gives a stable box; React fills `.current` with the DOM node after mount.
- The ref is created in the PARENT and passed down — in React 19 ref is a normal prop.
- Focus and scroll are the classic jobs state cannot do: reach for a ref, sparingly.

useRef — The DOM Escape Hatch, and What Is NOT State

Test it

The search box is focused the moment /courses loads and again after clicking the 🍰 Bakery chip; the hero button smooth-scrolls to the grid; and you can state the rule — redraw the screen means state, otherwise a ref — and point to a line in the app that follows it.

useContext and useReducer — State Without Prop Drilling

LAB 4.4

The theme toggle in the navbar and the shortlist count are needed six levels apart. The learner feels prop drilling, then moves the theme and the shortlist into React Context, guards each context with a custom hook that fails loudly, and names the shortlist's actions with a reducer.

You'll build

src/context/ThemeContext.jsx (dark mode) and src/context/CartContext.jsx (the shortlist), both consumed by Navbar

Tech & files: createContext, useContext, useReducer, provider composition, custom guard hooks

A pure reducer

FILE src/context/CartContext.jsx

```
// A reducer is PURE: (state, action) => nextState.
function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD':
      return state.some((c) => c.id === action.course.id)
        ? state
        : [...state, action.course]
    case 'REMOVE':
      return state.filter((c) => c.id !== action.id)
    case 'CLEAR':
      return []
    default:
      return state
  }
}
```

What this shows

- A reducer is a pure function: (state, action) => nextState, with no mutation.
- Each case returns a brand-new array, making it trivial to reason about and test.

A context provider

FILE src/context/CartContext.jsx

```
export function CartProvider({ children }) {
  const [items, setItems] = useLocalStorage('cart', [])

  const value = {
    items,
    addItem: (course) => setItems((p) =>
      p.some(c => c.id === course.id) ? p : [...p, course]),
    removeItem: (id) => setItems((p) =>
      p.filter(c => c.id !== id)),
    count: items.length,
  }

  return (
    <CartContext.Provider value={value}>
      {children}
    </CartContext.Provider>
  )
}
```

What this shows

- The provider owns the shortlist and exposes it plus the functions that change it.
- count is DERIVED (items.length) — never stored, so it cannot drift from the array.

Consuming context with a custom hook

FILE src/context/CartContext.jsx

```
export function useCart() {  
  const ctx = useContext(CartContext)  
  if (!ctx) {  
    throw new Error('useCart must be used inside <CartProvider>')  
  }  
  return ctx  
}
```

```
// In Navbar, at any depth, no prop drilling:  
const { count } = useCart()  
<span className="badge">🛒 {count}</span>
```

What this shows

- useCart wraps useContext and throws if used outside a provider — a clear early error.
- Any component reads the shared shortlist with one line, no prop drilling.

useContext and useReducer — State Without Prop Drilling

Test it

The 🌙 toggle in the navbar switches the whole site between light and dark and survives a refresh; the 🛒 shortlist count updates from any page with no props threaded through; and calling `useCart()` outside `<CartProvider>` throws a clear, named error.

Custom Hooks — Sharing Logic, Not State

LAB 4.5

The same `useState` + `useEffect` pair now appears in three places. The learner extracts `useLocalStorage` and `useDebounce`, wires the debounce into the course search so it stops filtering on every keystroke, and kills the near-universal misconception that a custom hook shares state between components.

You'll build

`src/hooks/useLocalStorage.js` and `src/hooks/useDebounce.js`, with the course search debounced to 300 ms

Tech & files: Custom hooks, the 'use' naming rule, `useEffect` cleanup, `localStorage`

useLocalStorage custom hook

FILE src/hooks/useLocalStorage.js

```
// A use* function that calls other hooks.
export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const stored = window.localStorage.getItem(key)
      return stored !== null ? JSON.parse(stored) : initialValue
    } catch {
      return initialValue
    }
  })
  const set = (next) => {
    setValue((prev) => {
      const resolved =
        typeof next === 'function' ? next(prev) : next
      window.localStorage.setItem(key, JSON.stringify(resolved))
      return resolved
    })
  }
  return [value, set]
}
```

What this shows

- A custom hook is just a function named use* that calls other hooks.
- It returns [value, set] like useState, but mirrors the value into localStorage.
- Lazy initial state (() => ...) reads storage once, on mount — not every render.

Custom hooks share logic, not state

FILE src/context/ThemeContext.jsx · CartContext.jsx

```
// Two callers of the SAME hook, each with its own
// independent key and value — logic is reused, state is not.
const [theme, setTheme] = useLocalStorage('theme', 'light')
const [items, setItems] = useLocalStorage('cart', [])

// useDebounce is reused the same way, per-component:
const debounced = useDebounce(query, 300)
```

What this shows

- Each call to `useLocalStorage` gets its own state — the hook shares behaviour, not data.
- That is the whole point of a custom hook: package logic once, reuse it everywhere.

Custom Hooks — Sharing Logic, Not State

Test it

Typing 'sourdough' filters 300 ms after you stop, not on every keystroke; the shortlist and theme still survive a refresh; both hooks live in `src/hooks/`; and you can explain why two components using `useDebounce` do not share a value.

END OF PHASE 04

Cook & Bake Academy can now shortlist courses, keep them across refreshes, toggle the theme, and jump to the grid.

Every lab in this phase added to the SAME app — one project, growing, not a pile of throwaway demos.

Lunch Break

30 minutes

PHASE 05

Give It a Real Backend

Building Cook & Bake Academy — we add a serverless API over a live Neon Postgres database, with real accounts, hashed passwords and JWTs

Give It a Real Backend

THE STORY SO FAR

The 20 courses live in a JavaScript array and every enrolment vanishes when the tab closes.

WHAT WE BUILD NOW

We add a serverless API over a live Neon Postgres database, with real accounts, hashed passwords and JWTs.

THE IDEAS THIS PHASE NEEDS

To build it, you will learn the three-tier split, API routes, parameterised SQL, bcrypt and JWT auth.

Three tiers, and one hard rule

- React (browser) --fetch--> /api/* (Vercel functions) --sql--> Neon Postgres. Three tiers.
- The BROWSER NEVER TALKS TO POSTGRES. It has no driver, no connection string and no SQL.
- It knows one thing: how to call our own /api/* URLs. The functions decide what is allowed.
- Every security rule lives in the middle tier, on a server, where the user cannot edit it.
- Anything you enforce in the browser is a suggestion. Anything you enforce in /api/ is a rule.

REMEMBER The browser asks the API. Only the API touches the database.

```
React (browser)      src/lib/api.js
  | fetch('/api/courses')
  v
/api/* (Vercel function) api/courses/index.js
  | sql`select ... from courses`
  v
Neon Postgres        DATABASE_URL lives ONLY here
```

THINK OF IT LIKE...

The API is the counter in a bank — customers state what they want; only staff go into the vault.

Why the browser cannot hold the password

- Vite compiles any VITE_-prefixed variable straight into the JavaScript bundle it ships.
- Every visitor downloads that bundle. They can read it. There is no exception and no hiding it.
- DATABASE_URL contains the database password and grants full READ AND WRITE to your data.
- So DATABASE_URL must NEVER carry the VITE_ prefix. It is read only in api/, via process.env.
- If you ever 'fix' a connection error by renaming it VITE_DATABASE_URL, you have published the vault key.

REMEMBER A VITE_ variable is public. DATABASE_URL must never have that prefix.

```
# .env.local — the prefix is the whole security boundary
DATABASE_URL=postgresql://user:PASSWORD@ep-...neon.tech/db
JWT_SECRET=<32 random bytes>
# NO VITE_ prefix on either. Both are server-only.

// api/_lib/db.js — the ONLY place it is ever read:
export const sql = neon(process.env.DATABASE_URL)
```

THINK OF IT LIKE...

Putting DATABASE_URL in a VITE_ variable is printing your house key on the flyer and posting it to every visitor.

A serverless API route is a function

- A file in `api/` becomes a URL: `api/courses/index.js` serves GET `/api/courses`.
- It exports one default async handler(`req`, `res`) — no Express app, no server to keep running.
- Square brackets make it dynamic: `api/courses/[slug].js` matches `/api/courses/macaron-masterclass`.
- Vercel hands you the matched segment as `req.query.slug`, and the query string as `req.query` too.
- The status code IS the answer: 200 for a course, 404 for no such slug, 401 for no valid token.

REMEMBER One file, one URL, one async handler — and it holds the only DB connection.

```
// api/courses/[slug].js (real code)
import { sql } from '../_lib/db.js'

export default async function handler(req, res) {
  const { slug } = req.query ?? {}
  const rows = await sql`
    select id::int as id, code, slug, title, fee::float8 as fee
    from courses where slug = ${slug}
  `

  if (rows.length === 0) throw new HttpError(404, 'No course.')
  return res.status(200).json(rows[0])
}
```

THINK OF IT LIKE...

Each file in `api/` is a shopfront window: one URL, one handler, opened only when someone knocks.

Tagged templates defeat SQL injection

- `sql`...`` is a TAGGED TEMPLATE. You call it with backticks, not brackets — and that is the whole story.
- It LOOKS like string interpolation but is not: the SQL text and the values travel SEPARATELY.
- The driver sends ``where slug = $1`` plus the value. Postgres parses the SQL, THEN binds the value.
- So a slug of ``x`; drop table users; --`` is looked up as an absurd literal slug and finds nothing.
- The one thing that reintroduces the hole is building the string yourself. Never do `sql(`...${x}...`)`.

REMEMBER The value travels separately from the SQL, so it can never be parsed as SQL.

```
// SAFE — a tagged template. Value travels separately.  
await sql`select * from courses where slug = ${slug}`  
//      Postgres receives: where slug = $1  
//      and, separately:  ["macaron-masterclass"]
```

```
// CATASTROPHIC — never build SQL by concatenation:  
await sql(`select * from courses where slug = '${slug}'`)  
// slug = "x"; drop table users; --" now EXECUTES.
```

THINK OF IT LIKE...

A parameterised query is a form with boxes — Postgres reads the form first, then fills the boxes. Text in a box can never become an instruction.

Never store a password

- Store a bcrypt HASH, never the password. Hashing is one-way: the hash cannot be turned back.
- bcrypt is deliberately SLOW (cost 10, ~100ms), which makes brute-forcing a stolen table painful.
- It salts every hash automatically, so two users with the same password get different hashes.
- Verify with `bcrypt.compare()`, which re-hashes with the stored salt and compares in constant time.
- NEVER select `password_hash` into a response. Whitelist the fields you send; do not blacklist.

REMEMBER Hash with `bcrypt`, compare with `bcrypt`, and never let the hash leave the server.

```
// api/auth/signup.js (real code)
const passwordHash = await bcrypt.hash(password, 10)
await sql`
  insert into users (email, name, password_hash)
  values (${email}, ${name}, ${passwordHash})
  returning id::int as id, email, name, created_at
` // note what is NOT in RETURNING: password_hash.

// api/auth/login.js
const ok = await bcrypt.compare(password, user.password_hash)
```

THINK OF IT LIKE...

A hash is a fingerprint, not a photograph — you can check a match, but you can never reconstruct the face.

Trust the token, never the body

- A JWT is header.payload.signature. The payload is ENCODED, not encrypted — never put a secret in it.
- The signature is what makes it trustworthy: it is computed with JWT_SECRET, which only the server has.
- Edit one byte of the payload and the signature no longer matches, so jwt.verify() throws.
- So the user id comes from the VERIFIED token's `sub` claim — NEVER from req.body.userId.
- If POST /api/enrollments trusted req.body.userId, anyone with curl could enrol anyone.

REMEMBER The id comes from a verified signature, not from something the caller typed.

```
// api/_lib/auth.js (real code)
export function requireAuth(req) {
  const header = req.headers?.authorization ?? ''
  if (!header.startsWith('Bearer ')) throw new HttpError(401, '...')
  const token = header.slice('Bearer '.length).trim()
  const payload = jwt.verify(token, JWT_SECRET) // throws if forged
  return Number(payload.sub) // verified, not claimed
}
```

THINK OF IT LIKE...

A JWT is a tamper-proof wristband — the payload is readable by anyone, but only the door staff can make a real one.

Ownership is enforced in the SQL

- An id in the URL comes from the address bar. Anyone can change a 4 to a 5 and ask to delete it.
- So the id alone is never enough. Every statement carries TWO conditions, and the second is ownership.
- `where id = \${id} and user\_id = \${userId}` — the userId comes from the verified JWT, not the caller.
- A row you do not own simply does not match, so Postgres changes nothing and we answer 404.
- Get this wrong and you have an Insecure Direct Object Reference — one of the commonest real bugs.

REMEMBER The URL says WHICH row; the token says WHOSE. Both go in the WHERE clause.

```
// api/enrollments/[id].js (real code)
const userId = requireAuth(req) // from the JWT
const { id } = req.query ?? {} // from the URL

const rows = await sql`
  delete from enrollments
  where id = ${id} and user_id = ${userId}
  returning id::int as id
`
// ^^^^^^^^^^^^^^^^^^^^^^^^^ the real check
if (rows.length === 0) throw new HttpError(404, 'Not found.')
```

THINK OF IT LIKE...

An id in the URL is a request, not a permission — asking for locker 5 does not make locker 5 yours.

Every remote read has three states

- A remote read is never instant, so you must model loading, error and success as distinct states.
- Hold three pieces of state: the data, a loading flag, and an error value.
- Show a skeleton while loading, a message on error, and the data on success — never assume success.
- AI-generated code reliably writes the loading and success branches and forgets the error one.
- Always clear the flag in a finally, or one failed request leaves the UI spinning forever.

REMEMBER Model loading, error and success — the error branch is the one AI skips.

```
// src/hooks/useCourses.js (real code)
try {
  const data = await api.get('/courses')
  setCourses(data); setError(null)
} catch (err) {
  setError(err.message)      // the branch AI forgets
} finally {
  setLoading(false)         // or it spins forever
}
```

THINK OF IT LIKE...

Every fetch is a package delivery — it is either in transit, lost, or on your doorstep, and the UI must show all three.

fetch does not reject on 404

- fetch rejects its Promise only on a network failure, not on a 404 or a 500 HTTP response.
- A 404 is a successful delivery of a failure message, so the await resolves perfectly normally.
- Always check response.ok (true only for 200-299) before you read the body.
- Skip it and an error payload like {error: '...'} sails into your UI and renders as if it were a course.
- Carry the status on the thrown error so a 404 (no such course) is distinguishable from a 500.

REMEMBER Check response.ok before reading the body — fetch won't reject on 404.

```
// src/lib/api.js (real code)
const res = await fetch(`${BASE}${path}`, { ... })
const data = res.status === 204 ? null : await res.json()

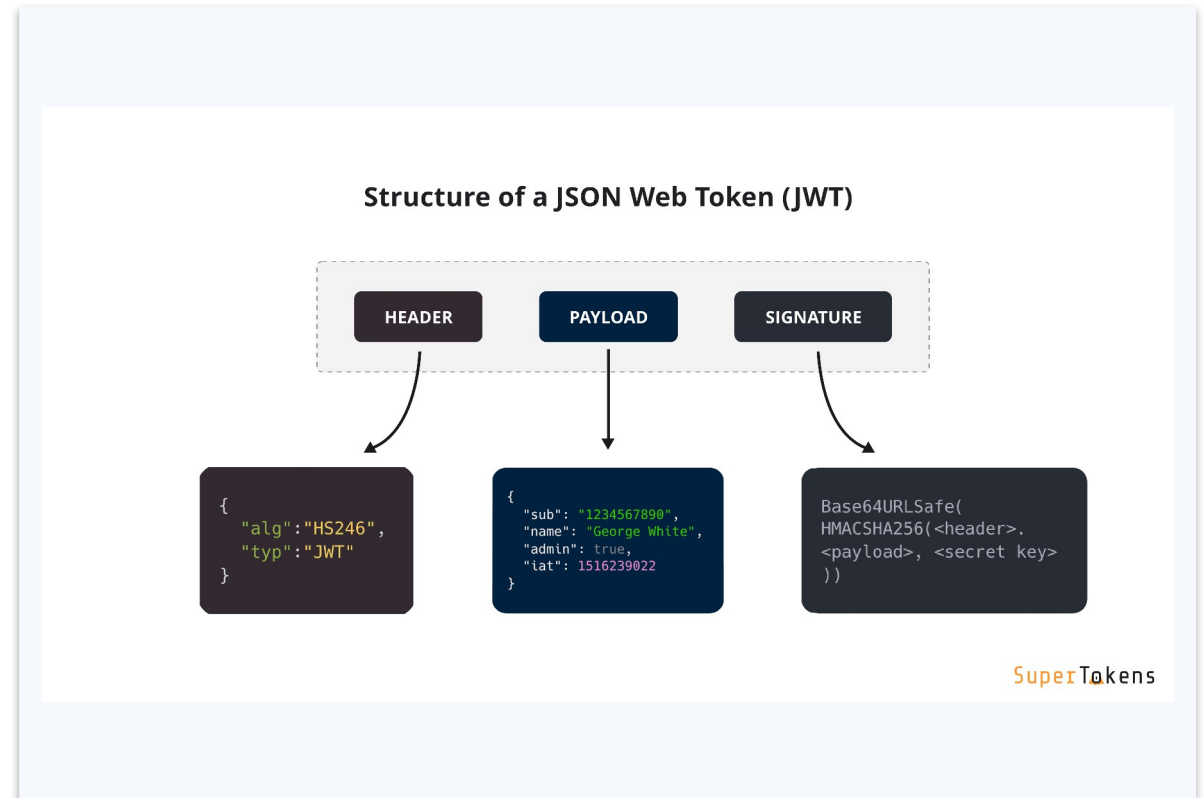
if (!res.ok) {
  const err = new Error(data?.error ?? `Failed (${res.status})`)
  err.status = res.status // 404 -> not found page
  throw err              // 500 -> error message
}
```

THINK OF IT LIKE...

fetch only cries if the road is out — a 404 is a signed 'not found' slip delivered successfully, so you must read the slip.

The structure of a JSON Web Token

- A JWT has three parts: a header, a payload of claims, and a cryptographic signature.
- The payload is only base64-encoded, not encrypted — anyone can read it, so never put a secret in it.
- The signature is computed with JWT_SECRET, known only to the server, so the token cannot be forged.
- Our API's requireAuth() verifies that signature and reads the user id from the 'sub' claim.
- That verified id — never a value from the request body — is what every protected route trusts.



A signed JWT carries the user's id in its payload; requireAuth() verifies the signature and reads 'sub'.

Three Tiers — A Neon Database and Your First API Route

LAB 5.1

The 20 courses live in a JavaScript array. The learner creates a Neon Postgres project, runs the real schema, and writes the first Vercel serverless function — GET /api/courses — reading it. The architectural rule is drilled here: the browser NEVER talks to Postgres, and DATABASE_URL must never carry a VITE_ prefix, because a VITE_ variable is compiled into the bundle every visitor downloads.

You'll build

A live Neon database with users, courses (20 seeded), enrollments and reviews — plus api/_lib/db.js and api/courses/index.js

Tech & files: Neon Postgres, @neondatabase/serverless, Vercel Functions, vercel dev, .env.local

Three tiers: the browser's only door

FILE src/lib/api.js

```
// The browser's ONLY way to reach the server.
// React (this file) --fetch--> /api/* --sql--> Neon
// No connection string, no password, no SQL in here.
const BASE = '/api'

async function request(method, path, body) {
  const token = getToken()
  const res = await fetch(`${BASE}${path}`, {
    method,
    headers: {
      ...(body ? { 'Content-Type': 'application/json' } : {}),
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    },
    body: body ? JSON.stringify(body) : undefined,
  })
  // ...check res.ok, parse JSON...
}
```

What this shows

- The browser knows one thing: how to call our own /api/* URLs. It never touches Postgres.
- The JWT rides along in the Authorization header; the server verifies it before trusting it.
- Every rule of security lives on the other side of this fetch, where the user cannot edit it.

fetch does not reject on 404

FILE src/lib/api.js

```
const data = res.status === 204
  ? null
  : await res.json().catch(() => null)

// fetch() does NOT throw on 404 or 500 — only a
// network failure rejects. Without this check, an
// error payload would sail into the UI as if it were
// data and be rendered as a course.
if (!res.ok) {
  const err = new Error(data?.error ?? `Failed (${res.status})`)
  err.status = res.status // carry it: 404 vs 500
  throw err
}
return data
```

What this shows

- fetch resolves a 404 normally — you must check `res.ok` yourself before reading the body.
- Carrying `err.status` lets a caller tell 'no such course' (404) from 'server broke' (500).
- This one wrapper gives every hook the same three-state behaviour for free.

Three Tiers — A Neon Database and Your First API Route

Test it

`curl /api/courses` returns all 20 Cook & Bake courses as JSON with numeric ids and fees; `?category=Bakery` returns 10; a POST returns 405; and grepping `dist/` for `'postgresql:/'` returns nothing.

Parameterised SQL — Dynamic Routes and an Injection Probe

LAB 5.2

Every course now needs its own URL. The learner adds the dynamic route `api/courses/[slug].js`, learns that Neon's `sql`` tagged template sends the `VALUE` separately from the SQL, and then attacks their own API with a real injection payload to watch it bounce — before deliberately writing the unsafe version and seeing the difference.

You'll build

`api/courses/[slug].js` — one course by slug, a real 404 when there is none — and a security probe you run yourself

Tech & files: Vercel dynamic routes, tagged templates, parameterised queries, HTTP status codes

The API route that reads the catalogue

FILE `api/courses/index.js`

```
import { sql } from '../_lib/db.js'

// GET /api/courses · public, no token required.
export default async function handler(req, res) {
  const { category, q } = req.query ?? {}
  const search = q ? `'%${q}%'` : null

  const courses = await sql`
    select id::int as id, code, slug, title, category,
           level, fee::float8 as fee, weeks, campus,
           summary, emoji, image
    from courses
    where (${category ?? null}::text is null
           or category = ${category ?? null})
           and (${search}::text is null
                or title ilike ${search})
    order by id
  `

  return res.status(200).json(courses)
}
```

What this shows

- A file in `api/` IS a URL: this one serves GET `/api/courses`, running on Vercel's servers.
- Query-string values are attacker-controlled — but they only travel as `${}` placeholders.
- `id::int` and `fee::float8` cast Postgres strings to real numbers, so `===` comparisons work.

Tagged templates defeat SQL injection

FILE api/_lib/db.js

```
import { neon } from '@neondatabase/serverless'

// DATABASE_URL is read ONLY here — no VITE_ prefix, so
// Vite never bundles it into the browser. The password
// never leaves the server.
export const sql = neon(process.env.DATABASE_URL)

// sql`...` is a TAGGED TEMPLATE: the query text and the
// value are sent to Postgres SEPARATELY as $1, $2...
// await sql`... where slug = ${slug}` // SAFE
// Building the string yourself is the door to injection:
// await sql(`... where slug = '${slug}'`) // NEVER
```

What this shows

- DATABASE_URL has NO VITE_ prefix, so it stays on the server and never reaches the bundle.
- The tagged template sends the SQL and the value separately — the value can't become SQL.
- A slug of `x`; drop table users; --` is looked up as an absurd literal and matches nothing.

Parameterised SQL — Dynamic Routes and an Injection Probe

Test it

GET /api/courses/artisan-sourdough-bread-baking returns one course; GET /api/courses/not-a-course returns 404 with a clear message; the injection payload returns a harmless 404 and the users table is still there; and grep finds no string-built SQL anywhere in api/.

Calling the API from React — Loading, Error and Success

LAB 5.3

The API works; the app still renders a hardcoded array. The learner writes the `src/lib/api.js` fetch wrapper — including the check every AI forgets, `response.ok`, because `fetch` does NOT reject on a 404 — then replaces the array with the `useCourses` and `useCourse` hooks and renders loading, error and success for real.

You'll build

`src/lib/api.js` and `src/hooks/useCourses.js` + `useCourse.js` — the catalogue now comes from Postgres

Tech & files: `fetch`, `response.ok`, HTTP status, `useState` + `useEffect`, skeleton and error UI

Signup: hash the password, never store it

FILE api/auth/signup.js

```
import bcrypt from 'bcryptjs'

// NEVER store the password. bcrypt is a slow, salted,
// one-way hash — a stolen table is useless to an attacker.
const passwordHash = await bcrypt.hash(password, 10)

const rows = await sql`
  insert into users (email, name, password_hash)
  values (${email}, ${name}, ${passwordHash})
  on conflict (email) do nothing
  returning id::int as id, email, name, created_at
`
// ^ password_hash is NOT in RETURNING. It never
// leaves this function.
return res.status(201).json({
  token: signToken(rows[0]), user: toClientUser(rows[0]),
})
```

What this shows

- bcrypt.hash is one-way and salts each hash, so identical passwords get different hashes.
- The RETURNING list omits password_hash — the hash is never sent in any response.
- toClientUser() whitelists the fields the browser may see, so nothing private leaks by accident.

Enrol: the id comes from the token

FILE api/enrollments/index.js

```
// THE important line. userId is the verified `sub` of
// the JWT — not req.body.userId, not a query param.
const userId = requireAuth(req)

const { courseId } = readBody(req)

// The caller chooses the COURSE. The caller does NOT
// choose the USER — we supply ${userId} from the token.
const rows = await sql`
  insert into enrollments (user_id, course_id, status)
  values (${userId}, ${courseId}, 'active')
  on conflict (user_id, course_id) do nothing
  returning id::int as id, course_id::int as course_id
`

if (rows.length === 0)
  throw new HttpError(409, 'Already enrolled.')
```

What this shows

- userId comes from requireAuth — a cryptographically verified value, not the request body.
- 'Enrol me in course 7' is safe; 'enrol anyone I like' is impossible because you can't set user_id.
- The unique (user_id, course_id) constraint means two fast clicks still create one row: a 409.

Ownership lives in the WHERE clause

FILE api/enrollments/[id].js

```
// An id in the URL is a REQUEST, not a PERMISSION.
const userId = requireAuth(req) // from the JWT
const { id } = req.query ?? {} // from the address bar

// Every statement carries BOTH conditions. A row you do
// not own simply does not match, so nothing is deleted.
const rows = await sql`
  delete from enrollments
  where id = ${id} and user_id = ${userId}
  returning id::int as id
`

if (rows.length === 0)
  throw new HttpError(404, 'Enrollment not found.')
```

What this shows

- The id says WHICH row; the token says WHOSE. Both go into the WHERE clause.
- Forget `and user\_id = \${userId}` and you have an Insecure Direct Object Reference.
- There is no RLS behind this — this WHERE clause IS the access control.

Calling the API from React — Loading, Error and Success

Test it

The catalogue renders from Postgres after a skeleton; editing a fee in Neon changes the page on refresh; /courses/not-a-course shows 'Course not found' (not a red error); and breaking the API base URL shows the error message instead of an endless skeleton.

Auth — bcrypt, JWT, and Ownership Enforced in SQL

LAB 5.4

Students must be able to sign up, sign in, enrol and review. The learner builds `api/auth/*` with bcrypt hashing and JWT signing, then the enrolments API — where the security rule of the whole course lands: the user id comes from the VERIFIED TOKEN, never from the request body, and ownership is enforced in the WHERE clause, not in an if statement.

You'll build

`api/auth/signup.js`, `login.js`, `me.js`, `api/_lib/auth.js`, the enrollments + reviews routes, and `src/context/AuthContext.jsx`

Tech & files: `bcryptjs`, `jsonwebtoken`, `JWT_SECRET`, Authorization: Bearer, insecure direct object references

AuthContext restores the session on boot

FILE src/context/AuthContext.jsx

```
const [user, setUser] = useState(null)
const [loading, setLoading] = useState(true)

// We may hold a token from a previous visit, but THIS
// page load has never had it checked. Ask the server.
useEffect(() => {
  async function restore() {
    if (!getToken()) { setLoading(false); return }
    try {
      const { user } = await api.get('/auth/me')
      setUser(user)
    } catch {
      clearToken() // expired or tampered with
    } finally {
      setLoading(false) // NOT optional — see below
    }
  }
  restore()
}, [])
```

What this shows

- The JWT lives in localStorage, so it survives a refresh — but the server must re-verify it.
- /api/auth/me verifies the signature and returns fresh user details, or 401s.
- `loading` is not optional: without it ProtectedRoute bounces a signed-in user on every refresh.

requireAuth: trust the signature, not the caller

FILE api/_lib/auth.js

```
export function requireAuth(req) {
  const header = req.headers?.authorization ?? ''
  if (!header.startsWith('Bearer '))
    throw new HttpError(401, 'Missing token.')

  const token = header.slice('Bearer '.length).trim()
  try {
    // Verifies the signature with JWT_SECRET (server-only).
    // Edit the payload and this throws.
    const payload = jwt.verify(token, JWT_SECRET)
    return Number(payload.sub) // the user id — verified
  } catch {
    throw new HttpError(401, 'Invalid or expired session.')
  }
}
```

What this shows

- The payload is only base64-encoded — the SIGNATURE is what makes the token trustworthy.
- `jwt.verify` throws on a bad signature or an expired token, so ``sub`` can be trusted.
- This returns an id the caller cannot forge, which every protected route builds its SQL on.

Auth — bcrypt, JWT, and Ownership Enforced in SQL

Test it

You can sign up, sign in, refresh and stay signed in; enrolling writes a row you can see in Neon; password_hash appears in NO API response; deleting another student's enrolment by id returns 404; and editing the JWT in localStorage signs you straight out.

END OF PHASE 05

Cook & Bake Academy can now read the catalogue from Postgres, sign up, enrol, review — and have it all still be there

tomorrow.

Every lab in this phase added to the SAME app — one project, growing, not a pile of throwaway demos.

PHASE 06

Ship a Real Product

Building Cook & Bake Academy — we add a multi-page app with course detail pages and a private student dashboard, deployed to Vercel — then you add a whole feature yourself

Ship a Real Product

THE STORY SO FAR

A powerful single page over a real database.

WHAT WE BUILD NOW

We add a multi-page app with course detail pages and a private student dashboard, deployed to Vercel — then you add a whole feature yourself.

THE IDEAS THIS PHASE NEEDS

To build it, you will learn routing, dynamic params, protected routes — and everything you have learned.

A Single Page Application

- A traditional site fetches a fresh HTML page from the server on every navigation.
- A single-page app loads one HTML document, then JavaScript swaps the view in place.
- The router also rewrites the URL bar so back, forward and bookmarks still work.
- No server round-trip means navigation is instant and React state is preserved across views.
- React Router maps each URL path to the component that should render for it.

REMEMBER One document; JS swaps views and rewrites the URL, no server reload.

```
// src/App.jsx (real code)
<Routes>
  <Route path="/" element={<RootLayout />}>
    <Route index element={<HomePage />} />
    <Route path="courses" element={<CoursesPage />} />
    <Route path="courses/:slug" element={<CourseDetailPage />} />
    <Route path="*" element={<NotFoundPage />} />
  </Route>
</Routes>
```

THINK OF IT LIKE...

A single-page app is a theatre with one stage — JavaScript changes the scenery and the marquee while the audience never leaves their seats.

Use `<Link>`, not `<a>`

- A plain `<a href>` triggers a full-page reload, refetching the whole app from scratch.
- That reload throws away every piece of React state — the shortlist, the search box, the scroll.
- `<Link to>` navigates within the SPA, swapping the view without any reload.
- The whole `CourseCard` is a `<Link>`, so clicking anywhere on a card routes to its detail page.
- Use `<a>` only for links that truly leave your app; use `<Link>` for every internal route.

REMEMBER `<Link>` navigates in-app and keeps state; `<a>` reloads and loses it.

```
// src/components/CourseCard.jsx (real code)
<Link to={`/${courses/${slug}}`} className="card">
  ...the entire card...
</Link>

<a href="/courses">Browse</a> // full reload: state gone
```

THINK OF IT LIKE...

An `<a>` tag rebuilds the whole theatre between scenes; `<Link>` just swaps the set, keeping every actor in place.

Layout routes and <Outlet/>

- A layout route is a parent route that renders shared chrome like the navbar and footer.
- Its child routes render inside it, so the shared frame is written once, not per page.
- The layout component renders <Outlet/> to mark where the matched child should appear.
- Layouts nest: DashboardPage is itself a layout, with My Courses and Profile inside its own Outlet.
- A PATHLESS parent route adds no URL segment — which is exactly how ProtectedRoute gates a subtree.

REMEMBER A layout route holds shared chrome; <Outlet/> is where children render.

```
// src/layouts/RootLayout.jsx (real code)
export default function RootLayout() {
  return (
    <>
      <Navbar />
      <main><Outlet /></main>  {/* the matched child */}
      <Footer />
    </>
  )
}
```

THINK OF IT LIKE...

A layout route is a picture frame with a window — <Outlet/> is the window where each page's picture slots into the same frame.

Dynamic params and useParams

- A path segment starting with a colon, like :slug, is a placeholder that matches any value.
- One route definition serves all 20 courses: /courses/macaron-masterclass, /courses/knife-skills...
- Inside the component, useParams() returns an object of the matched segment values.
- Read it with `const { slug } = useParams()`, then fetch that one course from `/api/courses/:slug`.
- `useNavigate()` moves programmatically: `navigate(-1)` goes back with no hardcoded path.

REMEMBER A :param matches any value; useParams() reads it inside the component.

```
<Route path="courses/:slug" element={<CourseDetailPage />} />
```

```
// src/pages/CourseDetailPage.jsx (real code)
const { slug } = useParams()
const { course, loading, error } = useCourse(slug)
// null course = no such slug -> render a real 404 page
```

THINK OF IT LIKE...

A dynamic segment is a mail slot labelled :slug — useParams reaches in and reads whichever course the URL dropped through.

URL state beats component state

- Filters, search terms and pagination are state, but they belong in the URL, not just useState.
- `/courses?category=Bakery&q=sourdough` is shareable, bookmarkable and survives a refresh.
- It also makes the browser back button restore the previous filter instead of leaving the app.
- `useSearchParams` reads and writes the query string exactly like a piece of state.
- Debounce before you write: one history entry per keystroke would wreck the back button.

REMEMBER Put shareable view state in the URL with `useSearchParams`, not `useState`.

```
// src/pages/CoursesPage.jsx (real code)
const [searchParams, setSearchParams] = useSearchParams()
const category = searchParams.get('category') ?? 'All'
const q = searchParams.get('q') ?? "
```

```
// { replace: true } updates the URL without a new
// history entry — the debounced value is what we write.
```

THINK OF IT LIKE...

Putting filters in the URL is writing them on a shareable ticket — bookmark it, send it, hit back, and the same view returns.

Protected routes wait for auth

- A protected route should render only for a signed-in user and redirect everyone else.
- The catch is that the session resolves asynchronously: on boot we ask `/api/auth/me` to verify the token.
- During that moment ``user`` is null but nobody is actually signed out. You know nothing yet.
- Guard on loading FIRST: while loading, show a message and redirect nobody.
- Skip that check and a signed-in student is bounced to `/login` on every single refresh.

REMEMBER Wait for loading before redirecting, or you eject real users on refresh.

```
// src/components/ProtectedRoute.jsx (real code)
const { user, loading } = useAuth()

if (loading) return <p>Checking your session...</p> // FIRST
if (!user) {
  return <Navigate to="/login" replace
    state={{ from: location }} /> // remember where
}
return <Outlet />
```

THINK OF IT LIKE...

Guarding before auth loads is checking a wristband in the dark — wait for the lights (loading) or you eject your own paying guest.

A client guard is UX, not security

- Hiding a link or redirecting from a route improves the experience for honest users.
- It is not security: anyone can edit the JavaScript, or skip the app entirely and call `/api/` with curl.
- The client cannot be trusted, because it runs entirely on the user's own machine.
- Real protection is in the API: `requireAuth(req)` for identity, and ``and user_id = ${userId}`` for ownership.
- Design so that a completely bypassed client guard exposes nothing the server would not already allow.

REMEMBER Client guards are UX only; the API's auth checks are what protect the data.

```
// Hiding the link is NOT security:  
{user && <NavLink to="/dashboard">Dashboard</NavLink>}
```

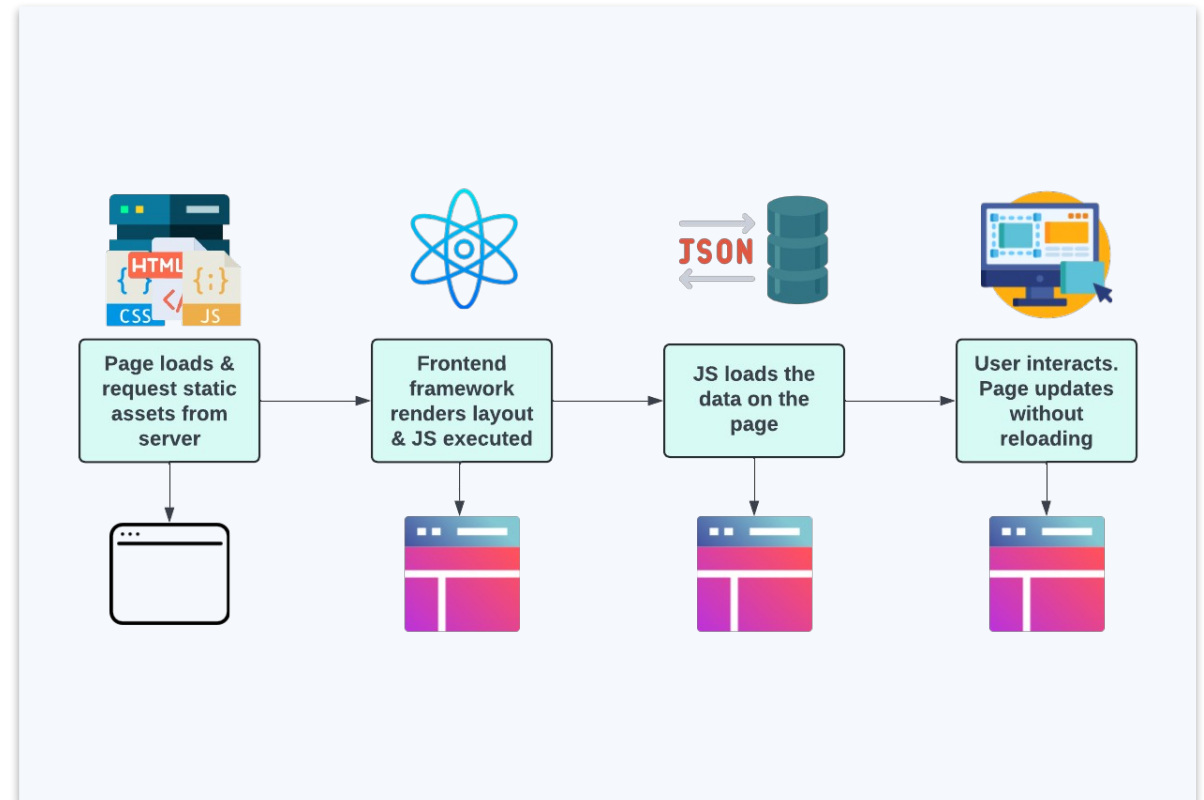
```
// THIS is security — and it runs on the server:  
const userId = requireAuth(req)    // 401 if forged  
await sql`select * from enrollments  
      where user_id = ${userId}`    // your rows only
```

THINK OF IT LIKE...

A client-side guard is a velvet rope, not a vault — anyone can step over it in devtools; only the API can lock the door.

How a single-page application loads

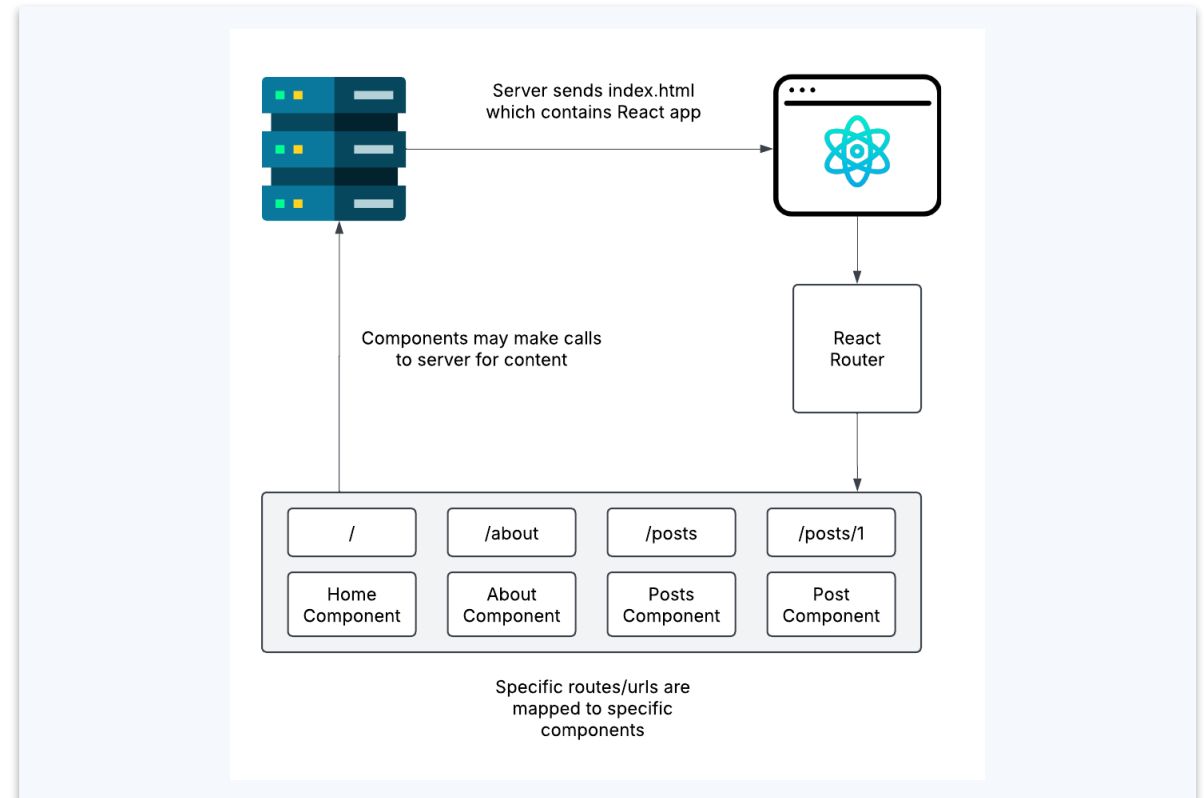
- The browser loads the page and its static assets from the server just once.
- The React framework then renders the layout and runs entirely in the browser.
- JavaScript fetches data as JSON and drops it into the already-rendered page.
- From there the user interacts and the view updates in place, with no full reload.



An SPA loads once, then updates the view in the browser without fresh page loads.

Client-side routing maps URLs to components

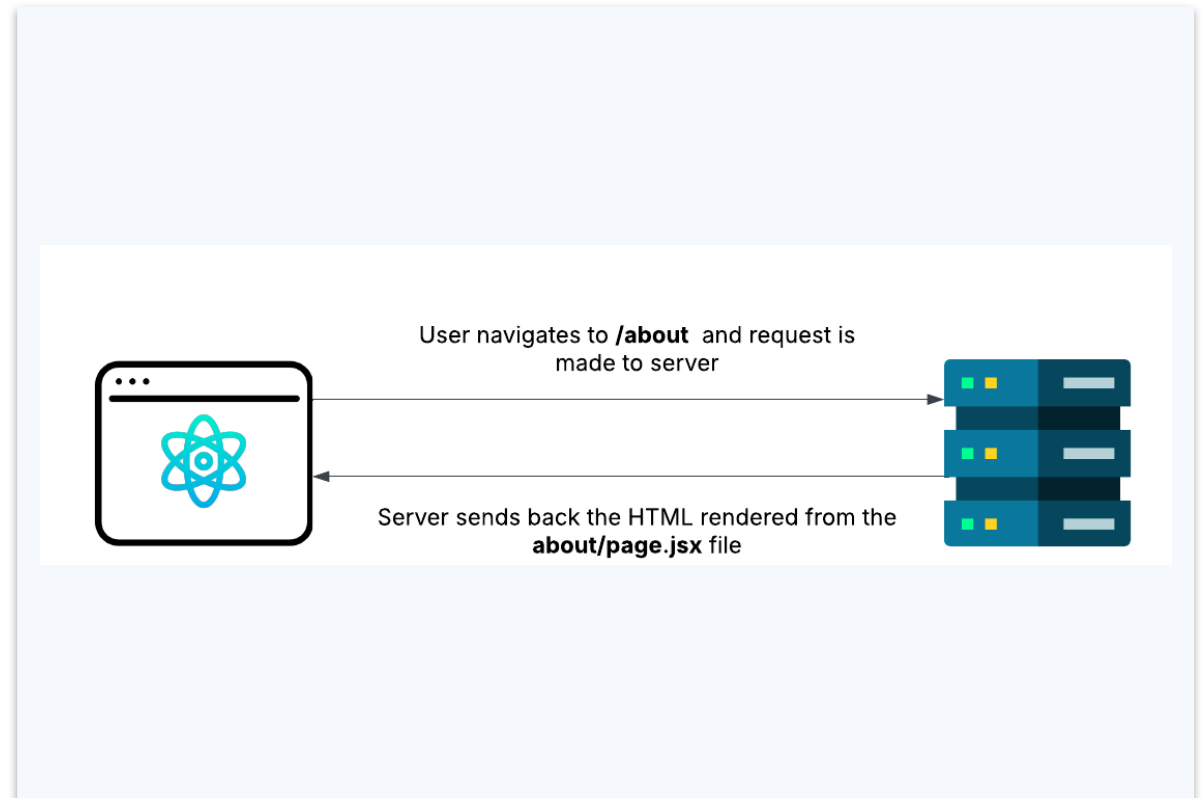
- The server sends one index.html that contains the whole React app.
- React Router inspects the current URL and decides which component to render.
- Each path — /, /about, /posts, /posts/1 — is mapped to its own component.
- Components may still call the server for data, but navigation stays in the browser.



React Router matches each URL path to a component, all without leaving the page.

Traditional server-side routing, for contrast

- In the traditional model, navigating to `/about` sends a fresh request to the server.
- The server renders the HTML for that page and sends the whole document back.
- Every navigation is a full round-trip, so any in-page state is lost on the way.
- From there the client-side JavaScript takes over and the page becomes interactive.
- Contrast this with the SPA model, where the client handles navigation after the first load.



Server-side routing round-trips for every page — the opposite of the SPA model.

Routes, Links and Layout Routes

LAB 6.1

Cook & Bake Academy is one long scroll. The learner installs React Router, splits it into Home, Courses and About, and builds a layout route whose `<Outlet/>` renders the matched page inside the shared navbar and footer.

You'll build

src/layouts/RootLayout.jsx with an `<Outlet/>`, plus `HomePage`, `CoursesPage` and `AboutPage`

Tech & files: react-router-dom, BrowserRouter, Routes, Route, Link, NavLink, Outlet

Declaring a route map

FILE src/App.jsx

```
import { Routes, Route, Navigate } from 'react-router-dom'
// App declares a route MAP. The layout route wraps its
// children; the matching URL renders into RootLayout's
// <Outlet/>. `index` is "/" itself.
export default function App() {
  return (
    <Routes>
      <Route path="/" element={<RootLayout />} />
      <Route index element={<HomePage />} />
      <Route path="courses" element={<CoursesPage />} />
      <Route path="courses/:slug"
        element={<CourseDetailPage />} />
      <Route path="about" element={<AboutPage />} />
    </Route>
  </Routes>
)
}
```

What this shows

- App declares routes, not a page; the matching URL renders into the layout's <Outlet/>.
- The layout <Route> wraps children; `index` is the route for '/' itself.

A layout route with <Outlet/>

FILE src/layouts/RootLayout.jsx

```
import { Outlet } from 'react-router-dom'
import Navbar from '../components/Navbar'
import Footer from '../components/Footer'
// The app shell. Navbar and Footer render on every
// page; the active route renders at Outlet.
export default function RootLayout() {
  return (
    <>
      <Navbar />
      <main style={{ minHeight: '70vh' }}>
        <Outlet />
      </main>
      <Footer />
    </>
  )
}
```

What this shows

- Navbar and Footer render on every page — the shared app shell.
- The active child route renders wherever <Outlet/> sits.

Routes, Links and Layout Routes

Test it

Clicking Courses and About swaps the page with no full reload, the navbar and footer never re-mount (the 🧺 count and the theme survive), and the active tab is highlighted.

Dynamic Routes, URL State and 404s

LAB 6.2

Each of the 20 courses needs its own shareable URL. The learner adds `/courses/:slug` backed by the `useCourse` hook and the `[slug]` API route from Topic 5, moves the search and category filters into the query string, and adds a real catch-all 404.

You'll build

CourseDetailPage at `/courses/:slug`, **NotFoundPage**, and shareable filter URLs like `/courses?category=Bakery&q=sourdough`

Tech & files: `useParams`, `useNavigate`, `useSearchParams`, catch-all routes, HTTP 404

Fetch one course by its slug

FILE src/hooks/useCourse.js

```
export function useCourse(slug) {
  const [course, setCourse] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)

  useEffect(() => {
    async function load() {
      try {
        const data = await api.get(`/courses/${slug}`)
        setCourse(data)
      } catch (err) {
        // A 404 is 'no such course', not a crash:
        if (err.status === 404) setCourse(null)
        else setError(err.message)
      } finally { setLoading(false) }
    }
    load()
  }, [slug])
  return { course, loading, error }
}
```

What this shows

- The [slug] dependency refetches whenever the URL segment changes.
- A 404 becomes course:null with NO error — which powers a real 'Course not found' page.
- A 500 sets error — two different failures, two different screens.

useParams reads the URL segment

FILE src/pages/CourseDetailPage.jsx

```
export default function CourseDetailPage() {
  // useParams reads :slug from the URL.
  const { slug } = useParams()
  const navigate = useNavigate()
  const { course, loading, error } = useCourse(slug)

  if (loading) return <Skeleton />
  if (error) return <p className="error">{error}</p>
  // null course -> no such slug -> render a 404.
  if (!course) return <NotFound slug={slug} />

  return (
    <article className="detail">
      <h1>{course.emoji} {course.title}</h1>
      <button onClick={() => navigate(-1)}>← Back</button>
    </article>
  )
}
```

What this shows

- useParams() reads the :slug segment declared in the route path.
- A null course means the slug matched nothing — render a 404 instead of crashing.
- navigate(-1) goes back to wherever the user came from, with no hardcoded path.

Dynamic Routes, URL State and 404s

Test it

Clicking a course card opens `/courses/<slug>` with its fee, weeks and campus; `/courses/does-not-exist` renders 'Course not found' and `/nonsense` renders the 404 page; and pasting `/courses?category=Bakery&q=sourdough` into a new tab restores the filtered grid.

Protected Routes and the Student Dashboard

LAB 6.3

Enrolments are private. The learner builds a ProtectedRoute layout route over the AuthContext from Topic 5, nests My Courses and Profile inside a dashboard layout, returns the student to the page they were trying to reach after login — and learns exactly why a client guard is not security.

You'll build

src/components/ProtectedRoute.jsx and a nested /dashboard with /dashboard/my-courses and /dashboard/profile

Tech & files: Navigate, useLocation, nested layout routes, Outlet, redirect-after-login

A route guard that waits for auth

FILE src/components/ProtectedRoute.jsx

```
import { Navigate, useLocation, Outlet }
  from 'react-router-dom'
import { useAuth } from '../context/AuthContext'
export default function ProtectedRoute() {
  const { user, loading } = useAuth()
  const location = useLocation()
  // MOST IMPORTANT LINE: wait for loading. Skip it and
  // a refresh bounces a signed-in user to /login.
  if (loading) return <p>Checking your session...</p>
  if (!user) {
    return <Navigate to="/login" replace
      state={{ from: location }} />
  }
  return <Outlet />
}
```

What this shows

- The most important line waits for `loading` — skip it and a refresh bounces a user.
- With no user, <Navigate> redirects to /login, remembering where they wanted to go.
- This is UX, not security — the real check is requireAuth() in every /api/ route.

Nested & auth-gated routes

FILE src/App.jsx

```
<Routes>
  <Route path="/" element={<RootLayout />}>
    <Route index element={<HomePage />} />
    <Route path="courses/:slug"
      element={<CourseDetailPage />} />
    <Route path="login" element={<LoginPage />} />
    {/* Pathless route: everything inside is auth-gated. */}
    <Route element={<ProtectedRoute />}>
      <Route path="dashboard" element={<DashboardPage />}>
        <Route index element={
          <Navigate to="my-courses" replace />} />
        <Route path="my-courses" element={<MyCoursesPage />} />
        <Route path="profile" element={<ProfilePage />} />
      </Route>
    </Route>
    <Route path="*" element={<NotFoundPage />} />
  </Route>
</Routes>
```

What this shows

- Nesting mirrors the URL; children render into their parent's <Outlet/>.
- A pathless <Route element={<ProtectedRoute/>}> gates everything inside it.
- DashboardPage is itself a layout, with My Courses and Profile in its own Outlet.

Protected Routes and the Student Dashboard

Test it

Visiting `/dashboard/my-courses` signed out redirects to `/login`; after signing in you land back on `/dashboard/my-courses`; refreshing while signed in keeps you there; and bypassing the client guard shows an empty, 401'd page — no data leaks.

Capstone — Assemble the Full-Stack App

LAB 6.4

Everything is now built. The learner walks the finished Cook & Bake Academy against a rubric that maps each concept in the course to the file in their own project that proves it, runs the AI React Bug Checklist over the generated code, and builds it clean.

You'll build

The complete Cook & Bake Academy: catalogue, search, shortlist, theme, API, database, auth, dashboard and routing

Tech & files: React 19, React Router 7, Vercel Functions, Neon Postgres, bcrypt + JWT

Capstone — Assemble the Full-Stack App

Test it

The app builds with no errors or warnings, every rubric row maps to a real file in your project, and you can defend each architectural choice out loud — especially why the browser has no database credentials.

Deploy the Full-Stack App to Vercel and Neon

LAB 6.5

The learner ships the whole three-tier app. The front end builds to static files, the api/ folder becomes real serverless functions, and DATABASE_URL and JWT_SECRET are set as ENCRYPTED, SERVER-SIDE Vercel environment variables — never with a VITE_ prefix. The result is the live app at cookbake-academy.vercel.app.

You'll build

A live, public, full-stack Cook & Bake Academy — real Postgres, real accounts, real enrolments

Tech & files: Vercel, Vercel Functions, environment variables, vercel.json rewrites, Neon

Deploy the Full-Stack App to Vercel and Neon

Test it

The live URL loads the 20 courses signed out; a deep link survives a refresh; you can sign up, enrol and see it on the dashboard in production; and grepping the built bundle for the connection string or the JWT secret returns nothing.

Mini-Capstone — Add Course Reviews, On Your Own

LAB 6.6

The app is built and live. Now you add one brand-new END-TO-END feature — star-rated course reviews — mostly on your own. You write the prompts, read the generated code against everything this course taught you, correct it, and redeploy. This is the exam of the whole course: a new API route with parameterised SQL, ownership from the JWT, a custom hook, a controlled form, a derived average, and a git push that puts it in

You'll build

Reviews on the course detail page: a public list, a form only signed-in students see, a live average rating, one review per student per course — deployed

Tech & files: `api/reviews/*`, an UPSERT, a custom hook, controlled forms, derived state, Vercel redeploy

Derived average, never stored

FILE src/hooks/useReviews.js

```
export function useReviews(courseId) {
  const [reviews, setReviews] = useState([])
  // ... load() fetches /api/reviews?courseId=... ...

  // My own review, if any — switches the form between
  // "write" and "edit".
  const myReview =
    reviews.find((r) => r.user_id === user?.id) ?? null

  // DERIVED at render — never stored, so it can never
  // drift from the reviews array.
  const count = reviews.length
  const average = count
    ? Math.round(
        reviews.reduce((s, r) => s + r.rating, 0)
        / count * 10) / 10
    : null
  return { reviews, myReview, count, average, submit, remove }
}
```

What this shows

- count and average fall straight out of the reviews array at render time.
- Because they're derived, they can never drift from the underlying list.
- myReview compares r.user_id === user.id — the API casts ids to int so 3 === 3, not 3 === '3'.

An UPSERT: write or edit one review

FILE api/reviews/index.js

```
// POST /api/reviews — reading is public, writing needs
// a token, so requireAuth() is inside the POST branch.
const userId = requireAuth(req)
const { courseId, rating, body } = readBody(req)

// One statement handles write AND edit. The table has
// unique (user_id, course_id), so a second review from
// the same user UPDATES their existing row.
const rows = await sql`
  insert into reviews (user_id, course_id, rating, body)
  values (${userId}, ${courseId}, ${rating}, ${body.trim()})
  on conflict (user_id, course_id) do update
    set rating = excluded.rating, body = excluded.body
  returning id::int as id, rating, body, created_at
`
```

What this shows

- user_id is \${userId} from the token, so a request can only ever touch your OWN review.
- The UPSERT means the client need not know whether a review already exists — and it's race-free.
- rating and body are validated on the server before the query — the form's checks are UX only.

StarRating: read-only or interactive

FILE src/components/StarRating.jsx

```
// Read-only when given just `value`; interactive when
// also given `onChange`. Interactive stars are real
// <button>s — never divs with onClick.
export default function StarRating({ value, onChange }) {
  const stars = [1, 2, 3, 4, 5]
  if (!onChange) {
    return stars.map((n) => (
      <span key={n}
        className={n <= value ? 'star is-on' : 'star'}>★</span>
    ))
  }
  return stars.map((n) => (
    <button key={n} type="button"
      onClick={() => onChange(n)}>★</button>
    ))
}
```

What this shows

- Pass just `value` for a display; add `onChange` to make the stars clickable.
- Interactive stars are real <button>s so they stay keyboard-accessible.

Mini-Capstone — Add Course Reviews, On Your Own

Test it

Signed out you can read reviews but not write; signed in you can post exactly one review per course, edit it and delete it; the average on the detail page updates live; another account cannot delete your review (404 from the SQL, not from an if); and after git push the feature is live on your Vercel URL.

END OF PHASE 06

Cook & Bake Academy can now navigate real routes, guard the dashboard, deploy the lot, and extend it on your own.

Every lab in this phase added to the SAME app — one project, growing, not a pile of throwaway demos.



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

Vibe Coding

Scaffolded a React app with an AI agent — and learned to audit what it wrote.

2

Cloud Deployment

Versioned with Git, built for production and published to Vercel.

3

Core React

Components, JSX & Babel, the real DOM vs the Virtual DOM, props, lists, events.

4

React Hooks

useState, useEffect with cleanup, useRef, useContext, useReducer, custom hooks.

5

Backend & Database

A serverless API over Neon Postgres; fetch with three states; bcrypt + JWT auth.

6

Routing & Shipping

Nested layouts, dynamic params, protected routes — deployed live.

The AI React Bug Checklist

1

key={index}

Corrupts list state on reorder or delete. Use a stable id.

2

No error branch

Every remote read has three states. Agents write two.

3

useEffect with no cleanup

Timers and subscriptions leak. StrictMode exposes it.

4

Derived state in useState

If you can compute it during render, compute it.

5

Unparameterised SQL

`sql(`... '${x}'`)` invites injection. Always use the `sql`` tagged template.

6

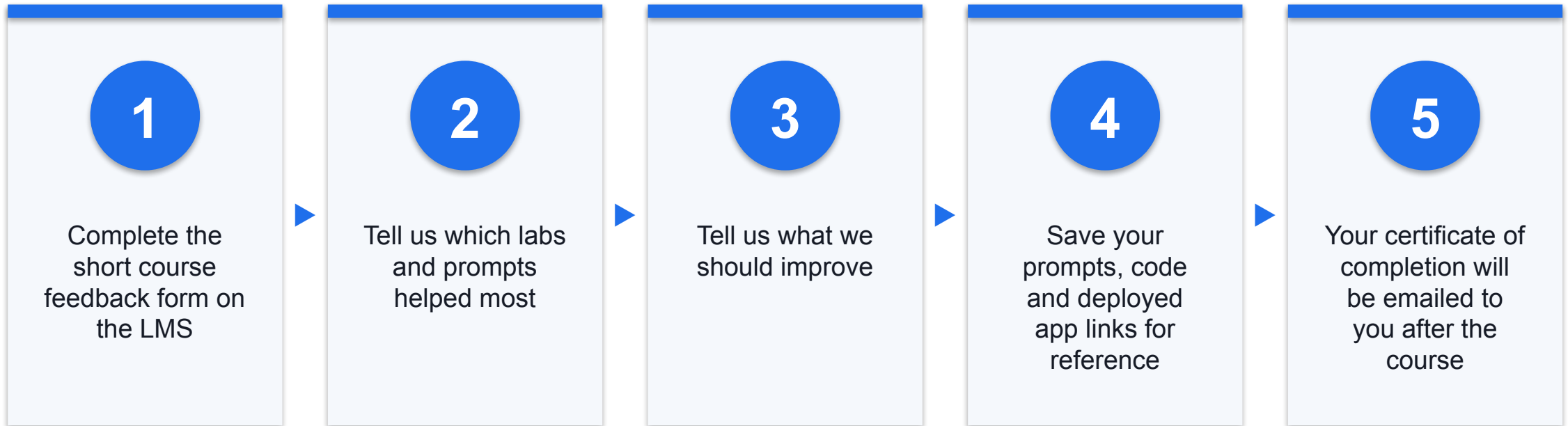
DATABASE_URL in the browser

A `VITE_` var ships to every visitor. Keep the connection string server-side.

Where to Go Next

- Redo the capstone from an empty folder, prompting only — then audit every file you generated.
- Add TanStack Query so you stop hand-rolling loading and error state in every hook.
- Add tests with Vitest and React Testing Library.
- Learn TypeScript, and generate types from your Neon schema.
- Explore React Router framework mode or Next.js when you need server rendering.

Course Feedback



HAPPY VIBE CODING

Thank You!

You can now build, understand and ship a full-stack React application — with an AI agent as your fastest colleague, not your blind spot.